POS — User Manual

For the people who run the store: cashiers, supervisors, managers, and the operator who installs the system. Covers the register and the back office.

Revision History

Version	Date	Changes
1.0	2026-07-22	First edition: register, back office, troubleshooting, FAQ, glossary.
1.1	2026-07-23	Added Chapter 14, End of Day (close, reopen, and what a closed day blocks). Later chapters renumbered.
1.2	2026-07-23	Chapter 14: explained why the next business date stays unavailable until the store's midnight, even after closing.
1.3	2026-07-27	Added Chapter 12, Payment methods (per-location tender groups and methods, what's archivable, and what reading the Z-report back can and can't tell you). Rewrote Chapter 5's tender passage for configurable payment methods. Later chapters renumbered.
1.4	2026-07-27	Chapter 11: added the On-screen keyboard toggle, the fields it covers, and why first-time activation of a keyboard-less terminal still needs a keyboard attached once.
1.5	2026-07-27	Chapter 7: added the back office's Variances queue — where to find which shifts need approval, and that it lists only, never approves.

1. Introduction

This manual covers everything you need to run the store day to day, and everything a manager needs to set it up. It's written for four kinds of reader, and you don't need to read the parts that aren't yours:

- **Cashiers** ring up sales and take payments.
- **Supervisors** do everything a cashier does, plus the actions that let money or stock leave the business without a customer noticing — voids, discounts, refunds, and approving a drawer that doesn't balance.
- **Managers** (admins) run the back office — the menu, staff, locations, and reports.
- **Operators** install and keep the system running.

The chapters are ordered the way the system is actually used: the overview comes first (Chapters 2–3), then the register — the till a cashier or supervisor stands at (Chapters 4–7, 15), then the back office a manager signs into from a laptop (Chapters 8–14). Troubleshooting, an FAQ, and a glossary sit at the back for when something goes wrong or a term is unfamiliar.

A few conventions hold throughout:

- Anything you tap or click — a button, a tab, a checkbox — appears in **bold**, exactly as it reads on screen: **Activate**, **Sign in**, **New user**.
- Money amounts are shown in pesos, because every screenshot in this manual comes from the same demo store in Manila — the figures and the prose always agree on what's on screen.
- **"Till"** means the same thing as **"register"** throughout this manual — a physical terminal, not a person. A store has several tills; each one is set up once and then used every shift after that.

2. System overview

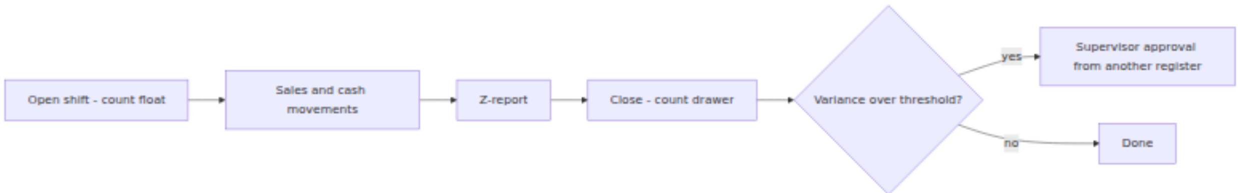
One order, two ways of selling

Underneath, every sale is the same thing: **open an order, add lines to it, take money, close it.** The system doesn't run two separate programs for retail and food service — it runs one order model and lets the till compress or expand it:

- **Retail** moves through that lifecycle in well under a minute, so the till collapses it into a single cart screen: scan, tap **Pay**, done.
- **Food service** lingers in the open phase for an hour or more, so that phase gets its own screen — a table, a running tab, courses fired to the kitchen — before the till ever asks for money.

A closed order is never edited. A refund doesn't erase or change the original sale — it adds new rows on top of it, so the receipt from the day of the sale still reprints exactly as it did that day, and the refund has its own paper trail.

Cash accountability follows the same shape, one level up: a till's shift opens with a counted float, runs sales and cash movements, and closes with another count.



If the counted drawer doesn't match what the system expects within a small threshold, closing records a **variance** that a supervisor has to approve — from a *different* till at the same location, never the one that just closed. Chapter 7 covers why.

The three surfaces

Surface	Who uses it	How it signs in
Register	Cashiers and supervisors, at a physical till	An activation code (once, per terminal — exchanged for a device token) plus a staff PIN (every shift)
Back office	Managers	An email and password — no till involved
API	The two apps above, and nothing else you should be talking to directly	Whatever the calling app is signed in as

The register and the back office are separate applications, on separate screens — a cashier's till never loads back-office code, and a manager's laptop never needs a device token. Both talk to the same server underneath, so a sale rung up at the till shows up in the back office's reports immediately, with nothing to sync or refresh by hand.

Roles

Two roles ship out of the box, deliberately coarse, and a manager can add more (Chapter 10):

- **Cashier** — rings up sales, opens and closes their own shift, takes payments.
- **Supervisor** — everything a cashier can do, plus voids, discounts, refunds, and approving a variance.

Both are assigned **per location** — the same person can be a cashier at one store and a supervisor at another, and the two never mix.

Admin sits outside the role system entirely: it's a property of the person, not a role tied to a location, so an admin is an admin everywhere, with full back-office access — catalog, staff, locations, reports, the audit trail. Short of full **Admin**, a manager can also grant someone a narrower slice of back-office access — a single permission, at a single location, without making them an admin (Chapter 10) — so "admin or nothing" is no longer the whole story on the back-office side.

Locations and the demo store

Every screenshot in this manual comes from the same seeded demo business — one company, three locations, all in the Asia/Manila timezone with prices that already include tax:

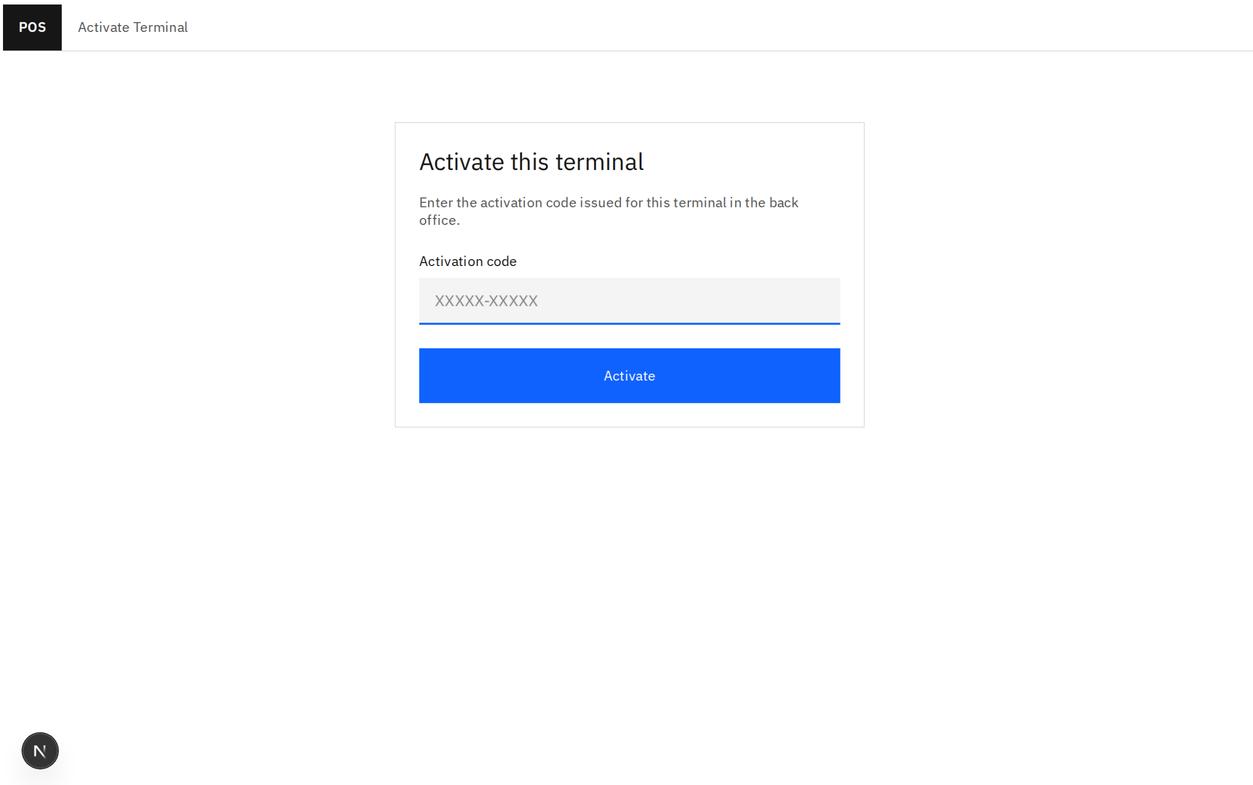
Code	Name
CAF	Manila Cafe
GRC	Manila Grocery
RST	Manila Restaurant

Manila Grocery runs retail (scan and go), Manila Restaurant and Manila Cafe run food service (tabs and tables). A location owns its own stock, its own registers, and its own staff assignments — nothing about a role or a stock count crosses from one location to another.

3. Getting started

Activate a register (once per terminal)

The first time a till is turned on, it shows an **Activate this terminal** screen and won't go any further until it's given an activation code.



The screen reads **"Enter the activation code issued for this terminal in the back office."** — a manager issues that code, not the person standing at the till. See [Issue an activation code](#) in Chapter 11 for how; it also covers a brand-new till's very first code, since there's no separate "first code" flow.

1. Get an activation code for this specific till from a manager.
2. Type it into **Activation code** (it's shaped like `XXXXX-XXXXX`).
3. Tap **Activate**.

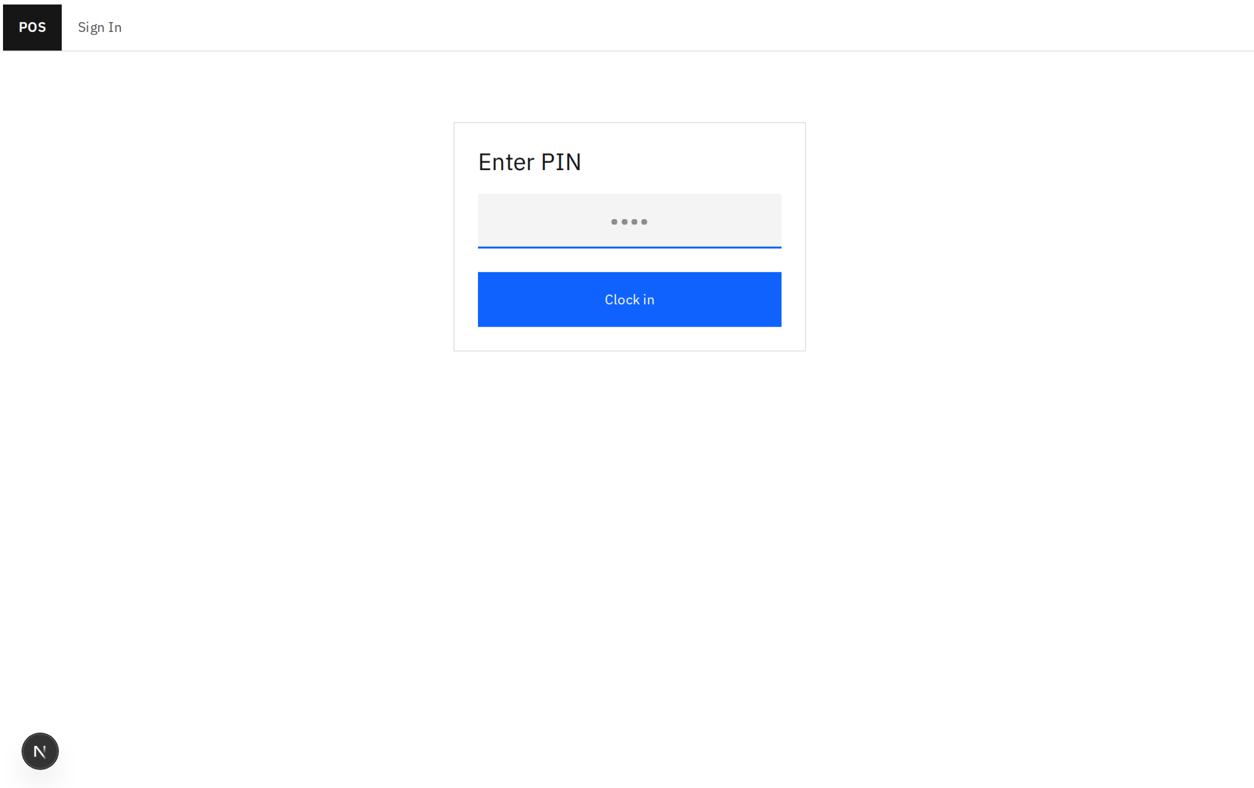
The code is **single-use** and **expires after 7 days** — it's a one-time credential the till exchanges for its own long-lived device token, not something typed in more than once. From that point on, the device token, never the code, is what proves this terminal's identity to the server.

Reissuing a code — for a lost till, or just as routine hygiene — kills the *old* device token and every staff session bound to it, in the same instant the new code is created. The old till doesn't get a grace period: it drops straight to a **Terminal**

disabled lockout screen the next time it talks to the server. Chapter 11 shows that screen (Figure 11.5) and walks through issuing a fresh code from the back office.

Clock in with a PIN

Once a till is enrolled, every shift starts the same way: an **Enter PIN** screen, shown whenever nobody's currently signed in.

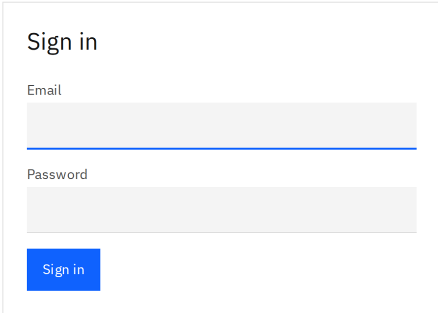


1. Type your 4–6 digit PIN.
2. Tap **Clock in**.

Five wrong PINs in a row locks that till out for 60 seconds. That's deliberate: PINs are short on purpose, and the lockout is what makes a short PIN safe to use at all.

Sign in to the back office

The back office has its own **Sign in** screen — an email and a password, nothing about a till or a device token involved.



1. Enter your **Email** and **Password**.
2. Tap **Sign in**.

Only admins can sign in here in this version — a wrong email, a wrong password, a deactivated account, and an account that isn't an admin all answer with the same message on purpose, so nobody can use the login screen to probe which email addresses exist in the system.

Common issues

Symptom	Cause	Fix
Activation code rejected	Expired (7 days) or already used	Get a fresh code from the back office (Chapter 11)
Till stuck on "Enter PIN", nothing happens	Five wrong PINs in a row	Wait 60 seconds, then try again
Back-office sign-in always says "incorrect"	Wrong email/password, account deactivated, or the account isn't an admin	All four cases look identical on purpose; ask another admin to check the account in Chapter 10

4. The register

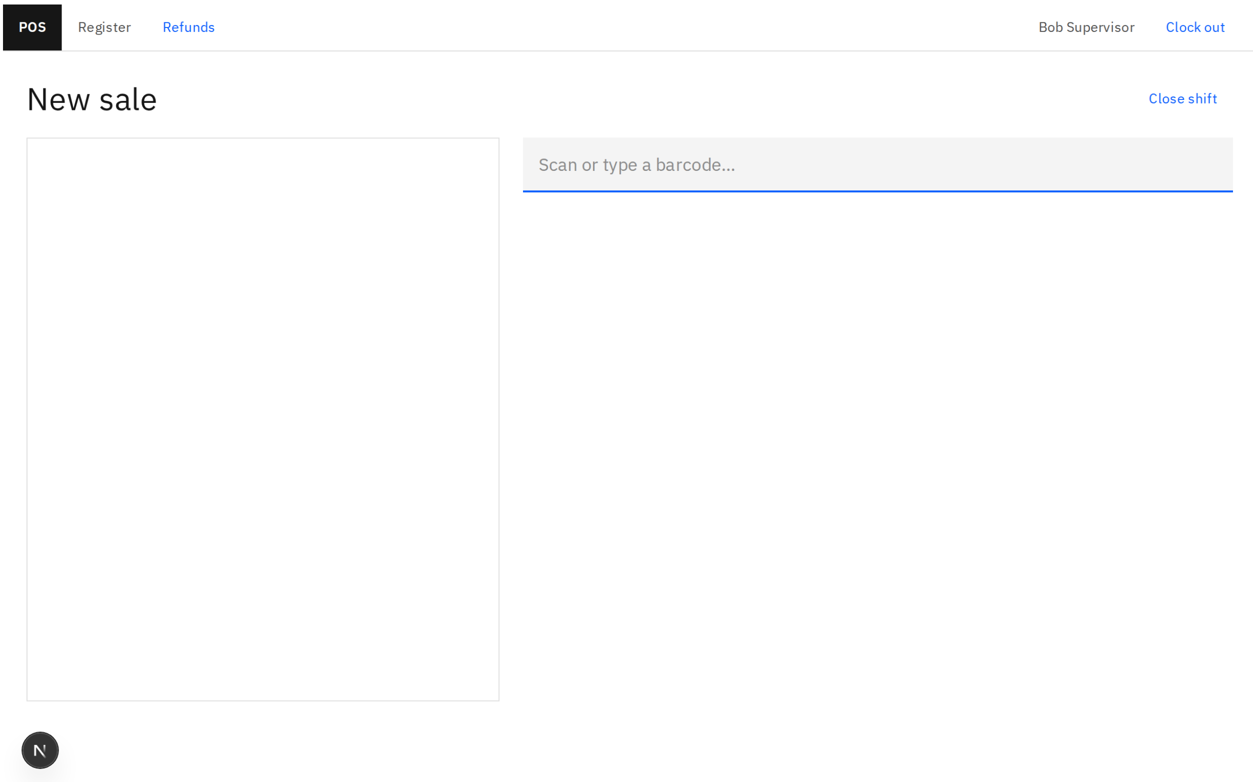
A till moves through the same four stages every day, and each stage is just whichever precondition isn't met yet — none of it is a setting:

1. **Activate this terminal** — until an activation code makes this terminal a register at all (Chapter 3).
2. **Enter PIN** — until somebody's clocked in (Chapter 3).
3. **Open shift** — until somebody's counted a float (Chapter 7).
4. A sale screen — the stage this chapter, and the next three, cover.

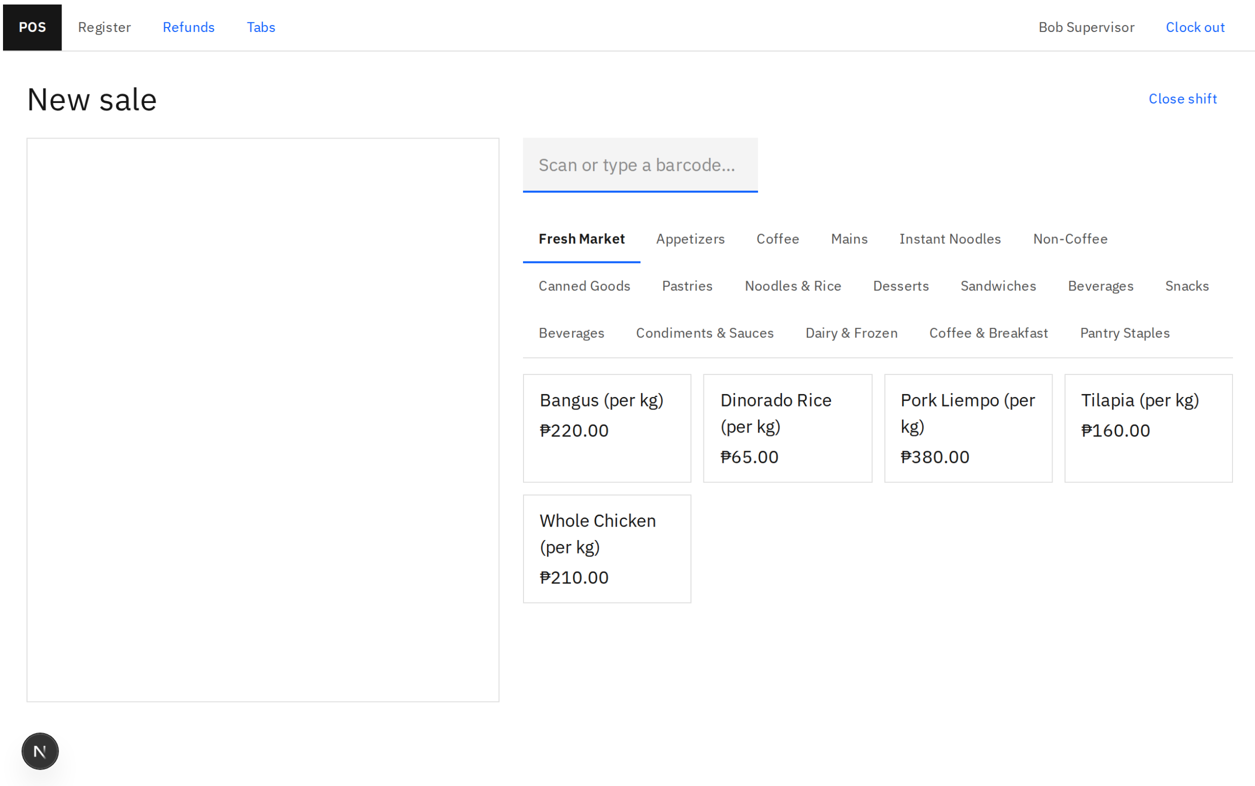
Retail or food

What that sale screen looks like is one setting a manager makes in the back office, not anything the till itself remembers — Chapter 11's Register mode section covers changing it.

- **Retail** mode opens straight onto a scan field: **New sale**, an empty cart, and **Scan or type a barcode…** waiting for input. Nothing to tap before the first item goes in — Chapter 5 covers this mode start to finish.



- **Food** mode opens onto a category rail and a grid of item tiles instead, and adds one more link — **Tabs** — next to **Register** and **Refunds** (the next section covers the rest of the top bar). Chapter 6 covers the floor and the menu grid start to finish.



There's no in-between and no per-shift toggle: a register is one mode or the other, every shift, until a manager changes it.

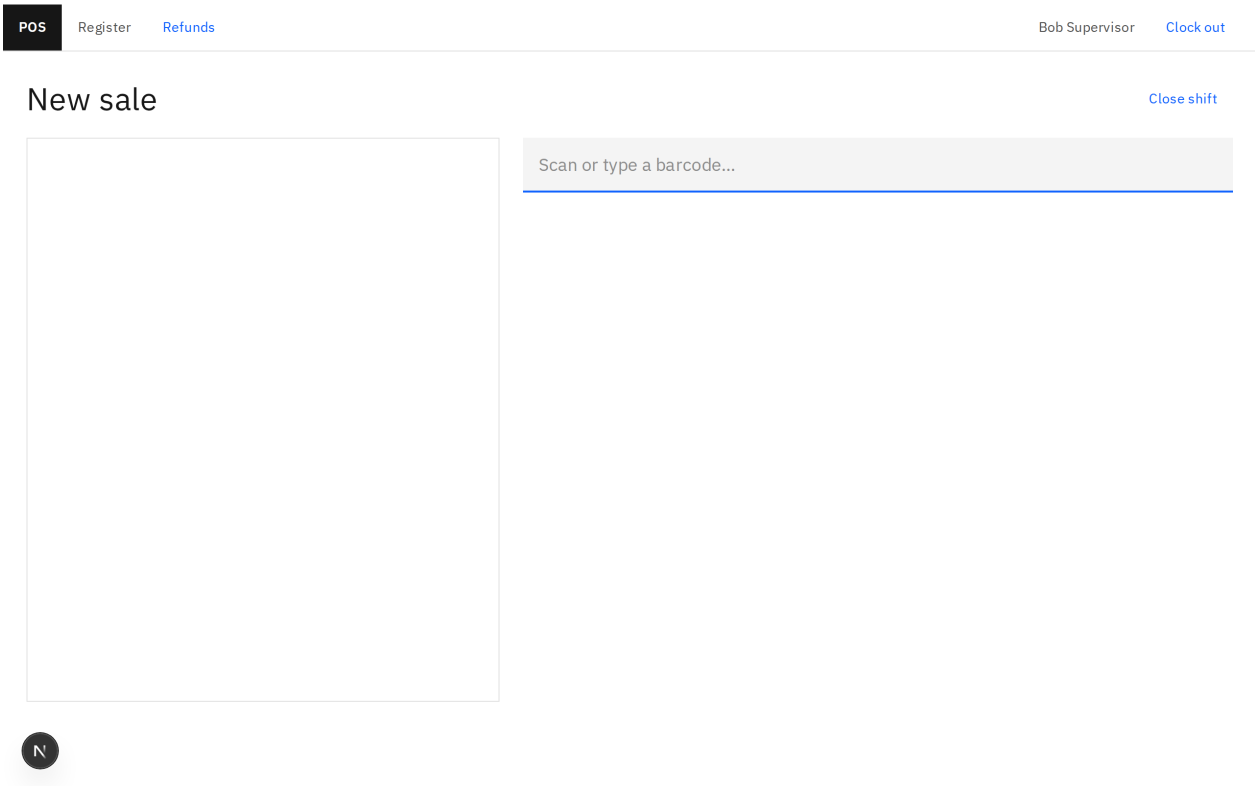
The top bar

Both figures above show the same header shape. On the left: **Register** and **Refunds** (visible only if you can refund — Chapter 5), plus **Tabs** on a food-mode till. On the right: the signed-in staff member's name and **Clock out** — which ends only that PIN session, not the shift; the drawer stays open for whoever clocks in next (Chapter 7). Just below that, the screen's own heading row carries **Close shift** on the right — a different button from **Clock out** above it, and available to whoever's clocked in right now, cashier or supervisor alike.

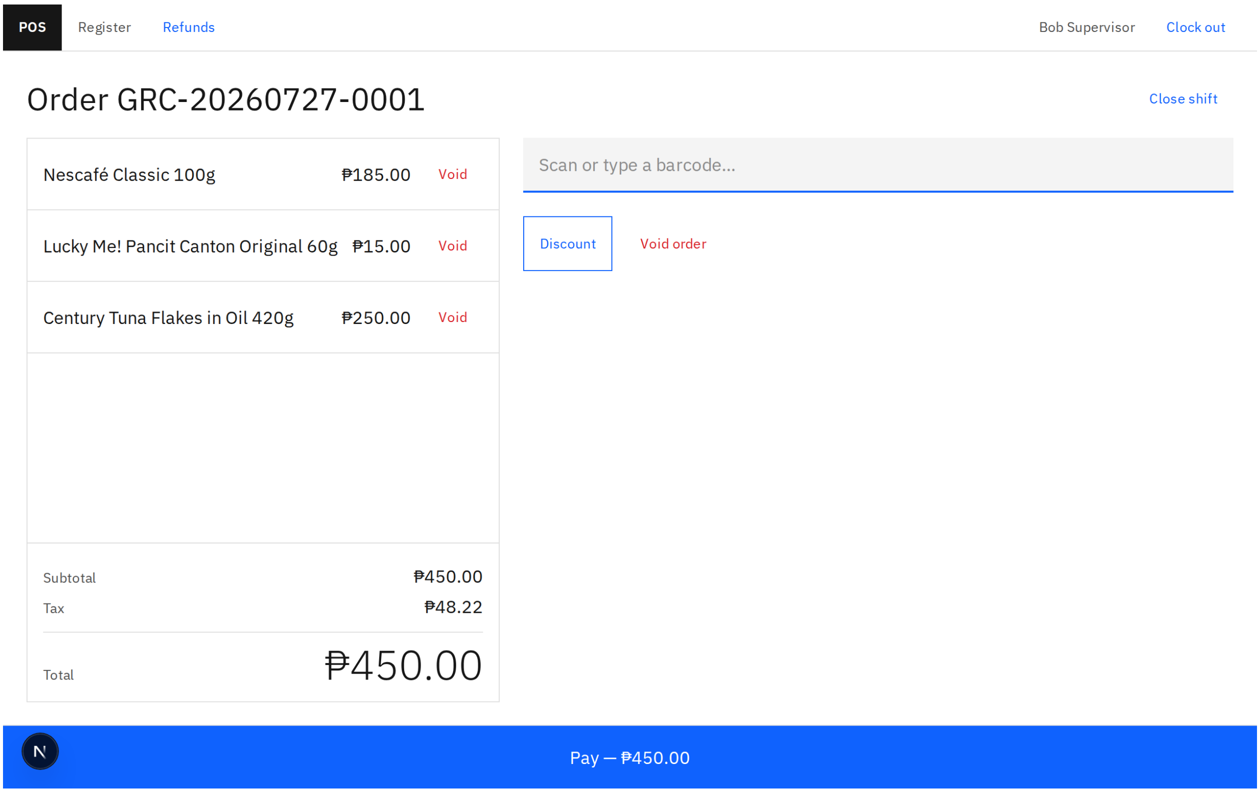
5. Retail selling

Retail mode collapses the whole order lifecycle into one screen: scan, **Pay**, tender, done. This chapter walks that path in order, using the same sale — rung up at Manila Grocery, Till 1 — that every figure below comes from.

Scan or type a barcode



1. Scan the item, or type its barcode into **Scan or type a barcode...** and press enter. The first scan opens the order for you — there's nothing to tap first.
2. Keep scanning. Each item lands in the cart below with its name and price; subtotal, tax, and total update as you go.



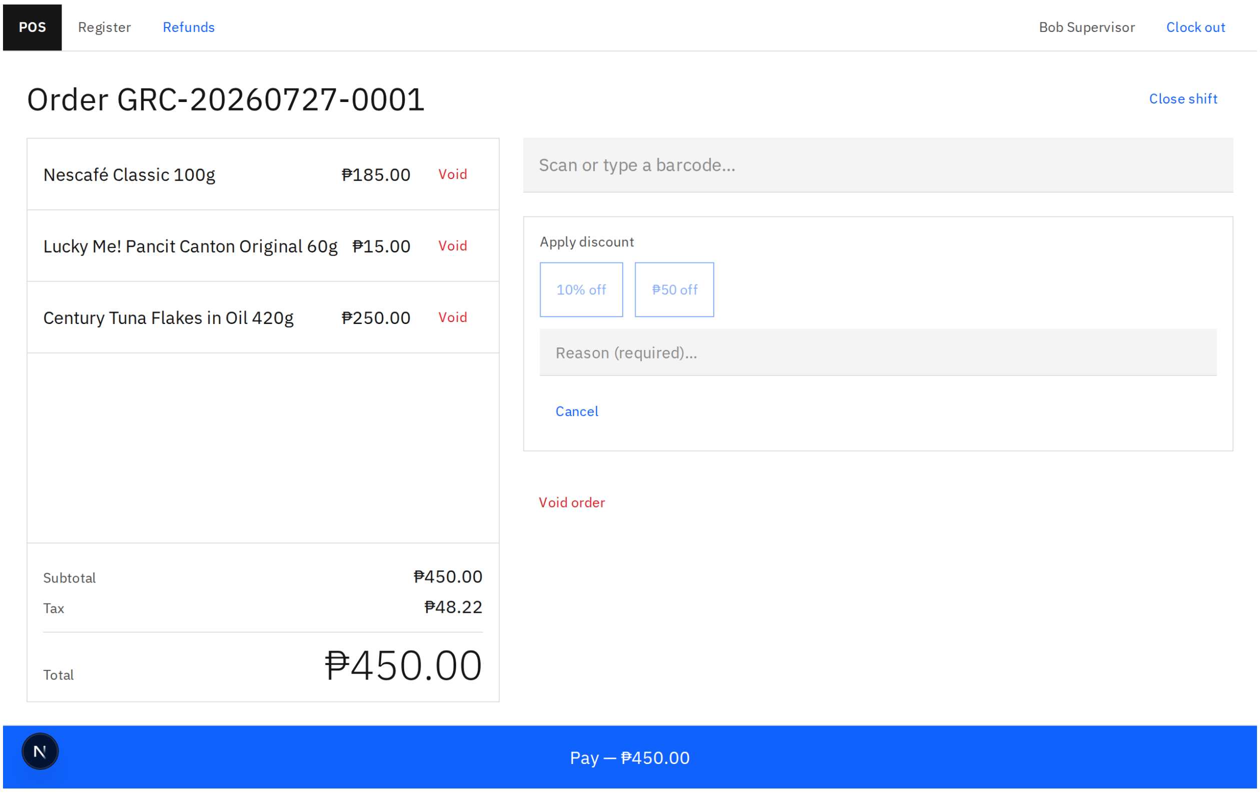
The catalog's Fresh Market category (Figure 4.2) prices several items **per kilogram** — Bangus, Dinorado Rice, Pork Liempo, Tilapia, Whole Chicken. A scan or a tap always adds exactly one unit of whatever's scanned, though — this version has no on-screen way for a cashier to type a weight in, so "Bangus (per kg)" rings up as 1 kg at ₱220.00 unless the quantity is corrected afterward, and there's no till button for that either (the same documented gap Chapter 6 covers for a fired food line's quantity). A cart line's quantity only shows on screen at all once it isn't a whole `1` — you see that happen in practice on a split check, in Chapter 6.

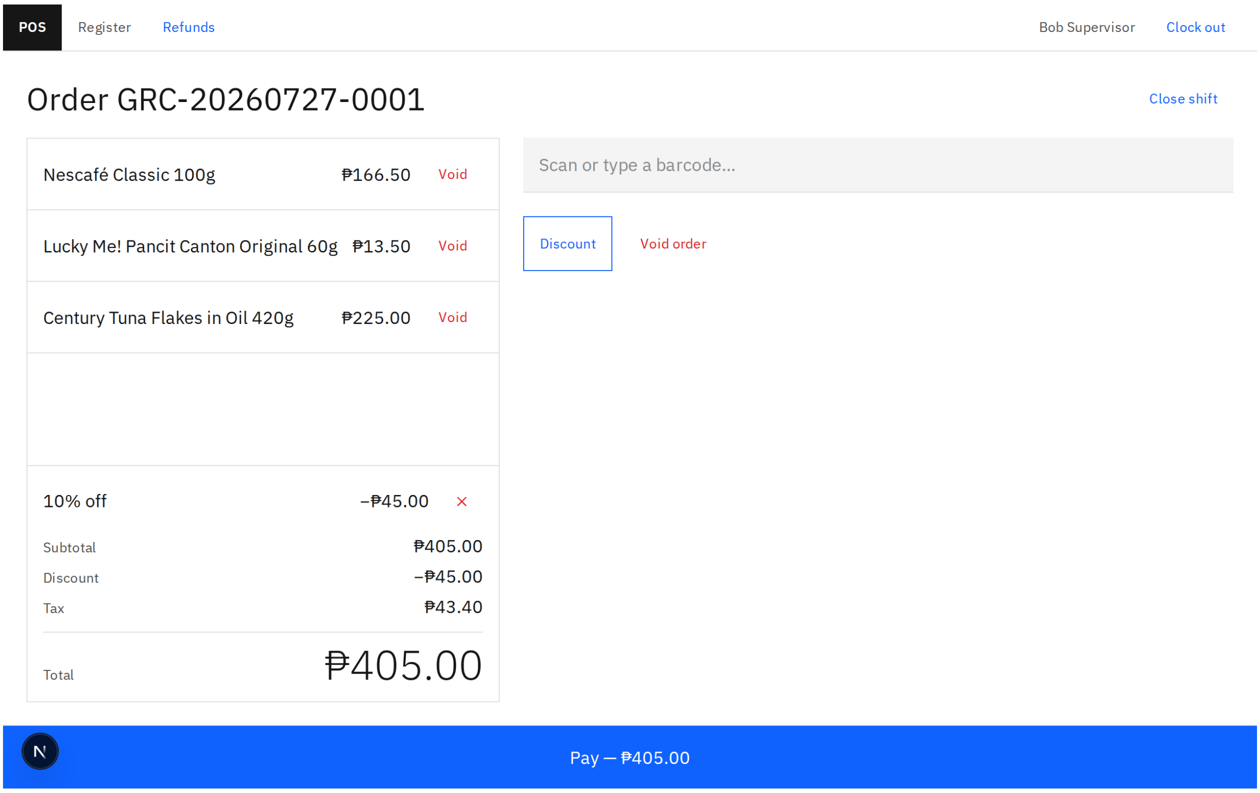
Void a line

Tap **Void** on any cart row to remove it, or **Void order** below the cart to remove every line at once. Both only appear for a supervisor's PIN session ([Roles](#), Chapter 2) — a cashier's own screen doesn't show either button at all.

Apply a discount

1. Tap **Discount**.
2. Type why into **Reason (required)...** — every discount stays disabled until you do.
3. Tap the discount by name to apply it to the whole order.





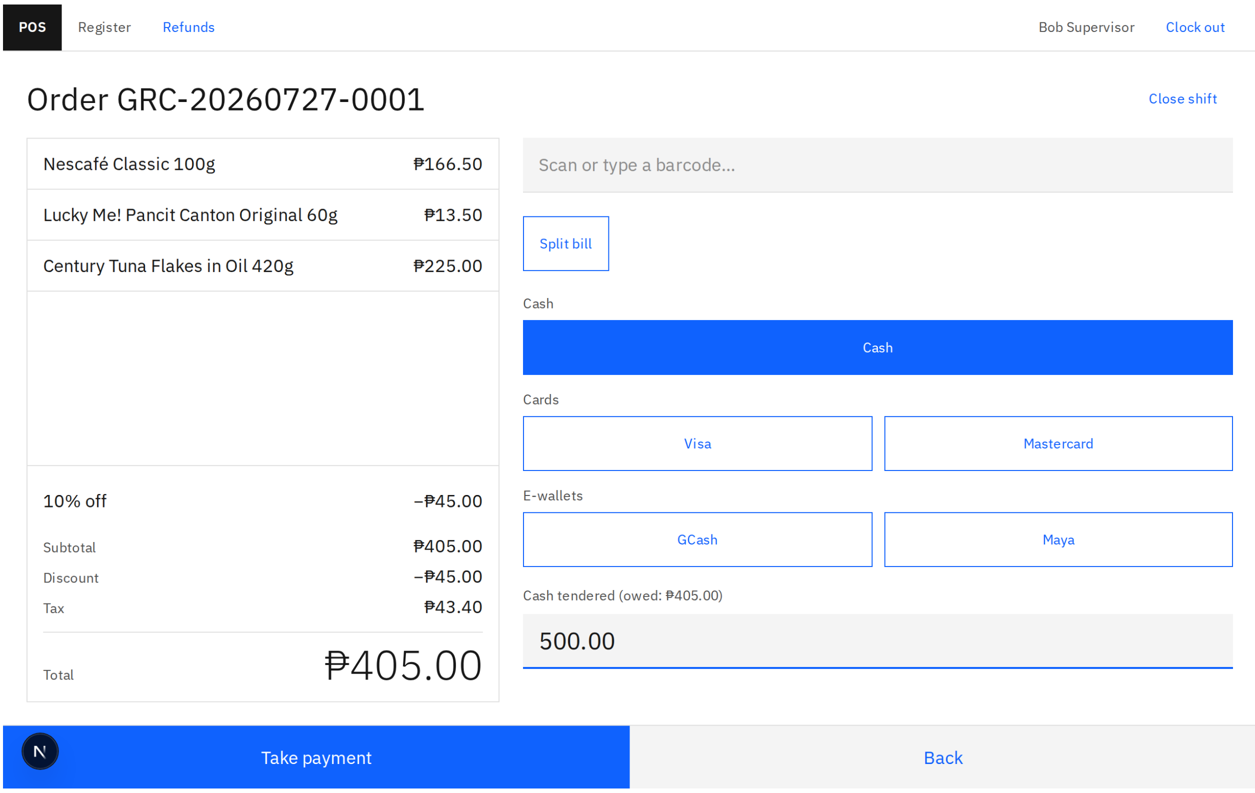
To remove one, tap the × next to it in the cart. Like voiding, **Discount** only appears for a supervisor. The back office's Discounts tab (Chapter 9) carries its own **Requires supervisor** flag per discount, but every discount already needs one today regardless of that flag, since only the supervisor role holds the permission to open the panel at all. A discount with that flag turned off is enforced as cashier-safe by the API, but the till screen has no way to reach it yet — the **Discount** button itself still renders for a supervisor's PIN session only.

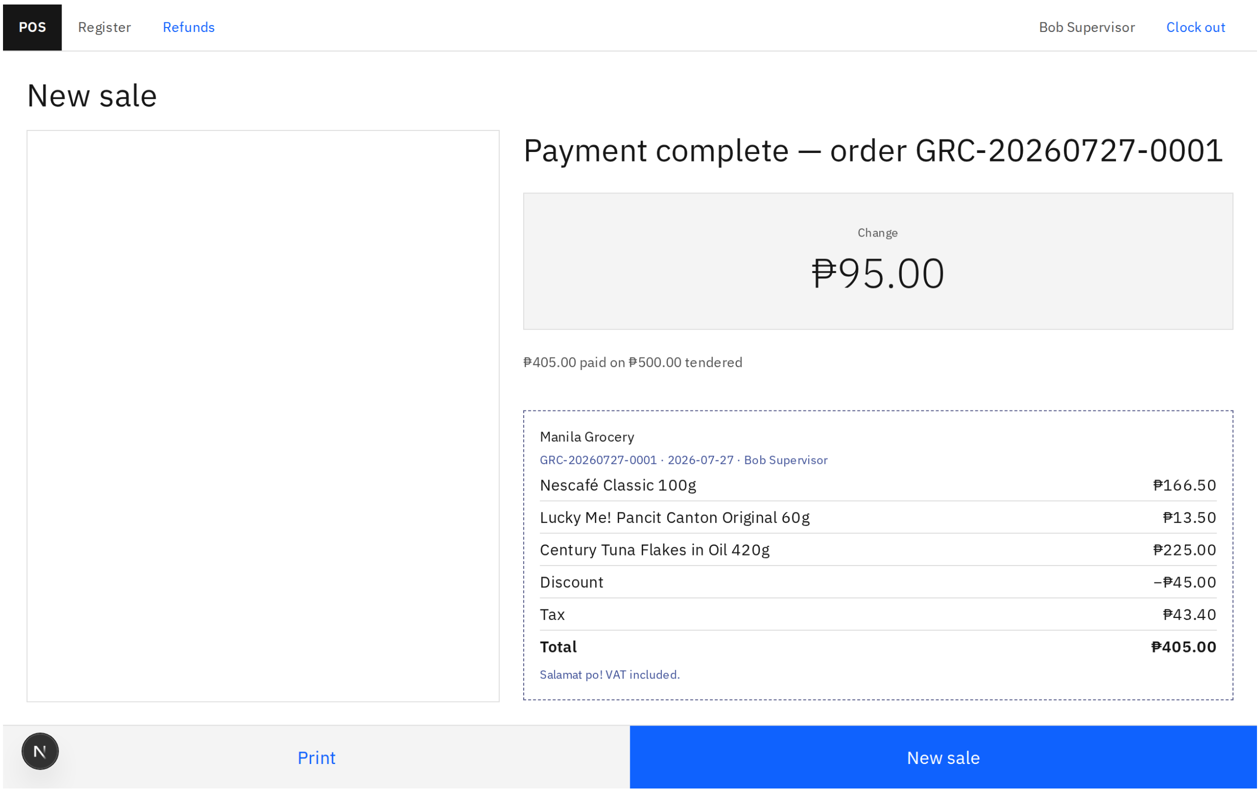
Pay, tender, and print

Tap **Pay — [amount]** (the button shows exactly what's owed).

The tender buttons are whatever your store set up (Chapter 12) — often **Cash**, a card scheme or two, and an e-wallet — grouped under headings like **Cards** or **E-wallets**. What you type next depends on which one you tap:

- **A cash method:** type what the customer handed you into **Cash tendered (owed: …)**, then tap **Take payment**. The next screen works out **Change** for you — never do that math yourself.





- **Anything else** (a card, an e-wallet, a voucher): type the reference the other machine gave you into **[method] reference (owed: …)**, then tap **Take payment**. This till only *records* what that machine already did — it doesn't talk to it.

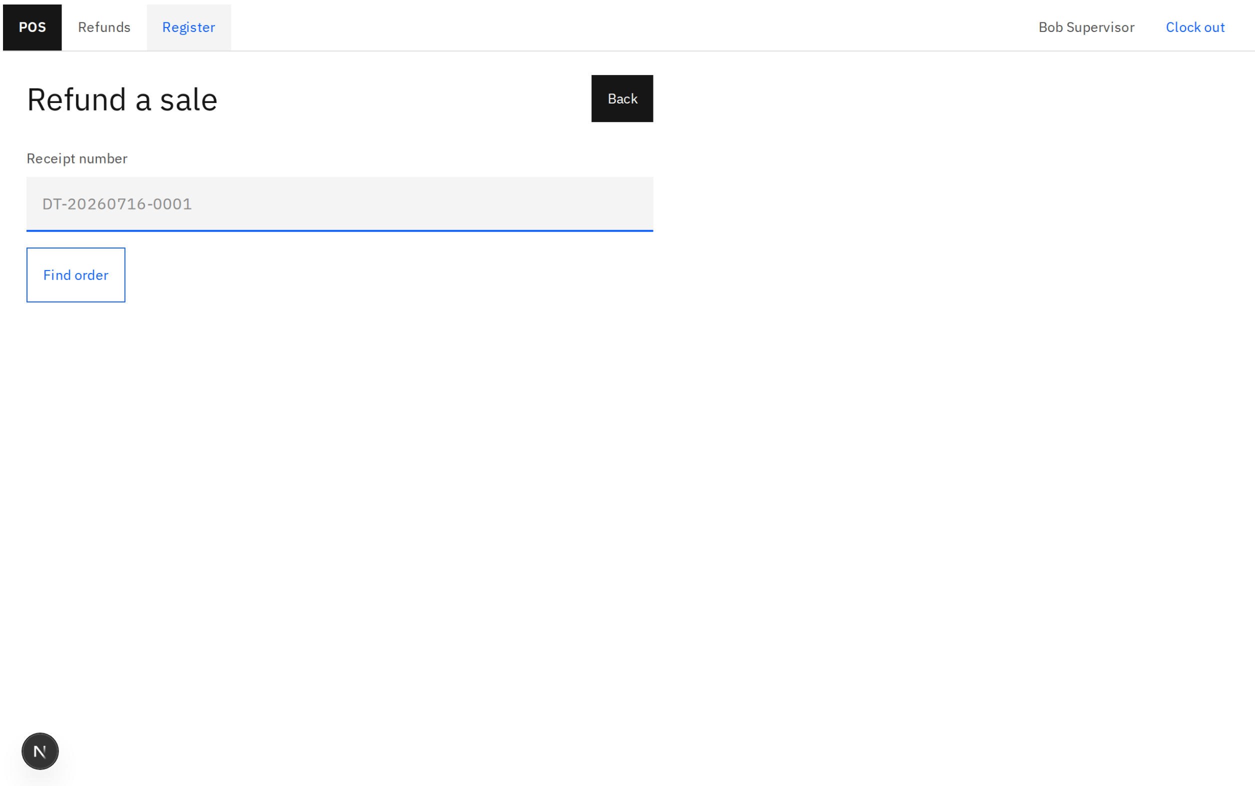
If the tender step says there are no payment methods, nobody has set any up for this location yet; an admin fixes that in Chapter 12.

Tap **Print** for a paper receipt, or **New sale** to move on. In the desktop shell, **Print** sends the receipt to the till's receipt printer; in an ordinary browser tab, it opens the browser's own print dialog — Chapter 16 covers both.

Refund a closed sale

Tap **Refunds** in the top bar (only visible if you can refund).

1. Type the receipt number into **Receipt number** and tap **Find order**.



1. For each line being returned, type the quantity to refund. Leave **Restock** checked to put that stock back, or clear it if the goods are damaged.
2. Type a **Reason** and tap **Refund cash**.

The refund always comes out of *this* drawer as cash, whatever the sale was originally paid with. Only a **closed** order can be refunded.

Common issues

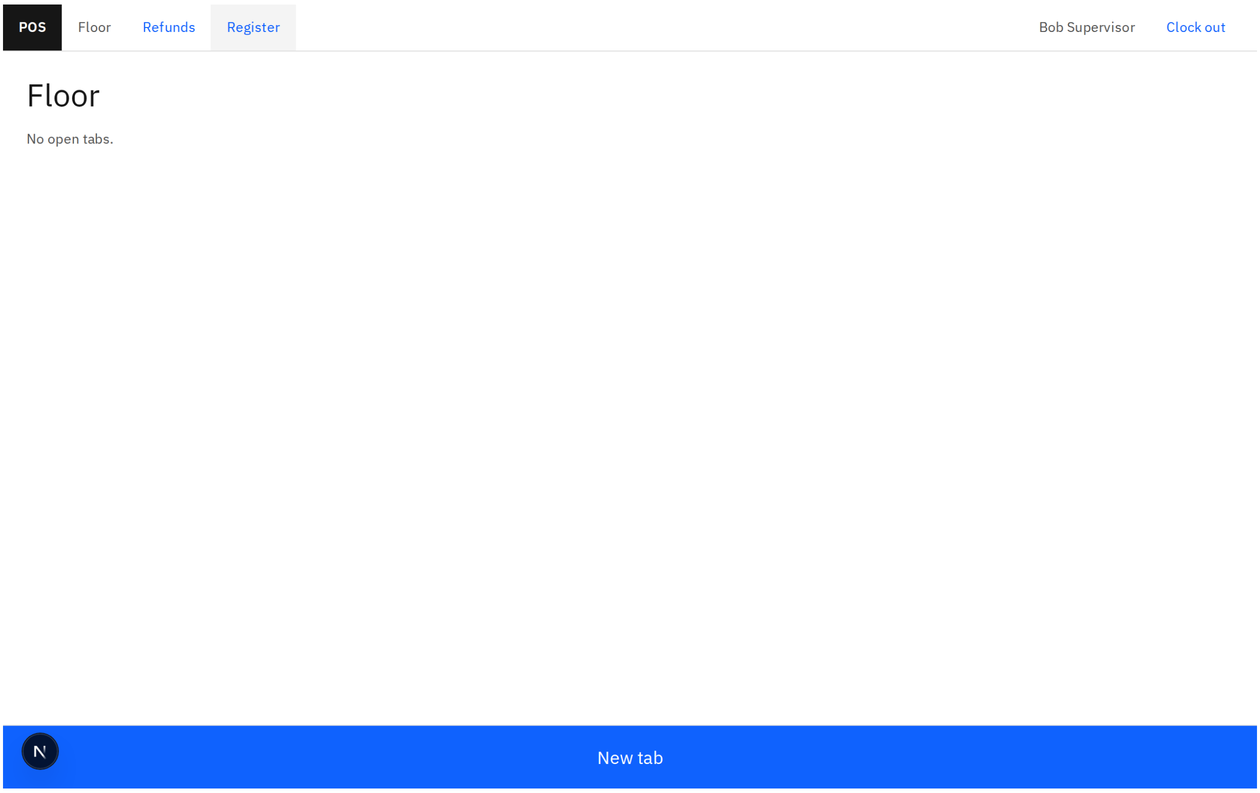
Symptom	Cause	Fix
"The requested resource does not exist." on a scan	The barcode doesn't match any active variant	Check the barcode, or find the item by its product/variant in the back office (Chapter 9)
No **Discount** or **Void** button on screen	You're signed in as a cashier — both only render for a supervisor	Ask a supervisor, or have them clock in to do it themselves (Roles, Chapter 2)
"This refund would return more than was sold on this line."	The typed quantity is more than what's left on that line, net of any earlier refund	Refund no more than what's left on the line

6. Food service

A food-mode till trades the scan-first screen for a floor of open tabs and a menu grid, because a table lingers in the open phase for an hour or more instead of closing in under a minute. Every figure below comes from Manila Restaurant, Till 1.

The floor and opening a tab

Tap **Tabs** to see every open tab at this location — the **Floor**. Tap **Register** to come back to the sale screen.

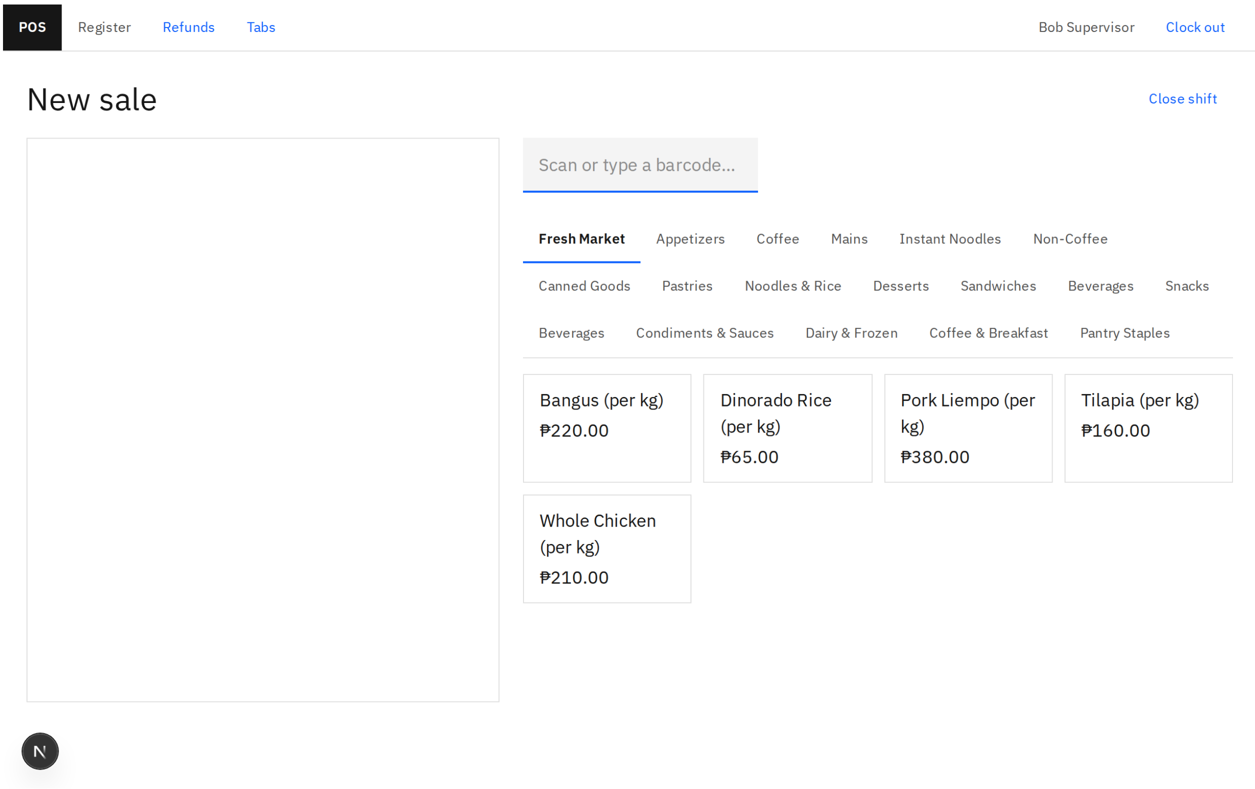


1. Tap **New tab**.
2. Optionally type the table into **Table (optional)…** — leave it blank for a walk-in or a bar tab.
3. Tap **Open tab**.

You land on the sale screen with an empty order already seated at that table.

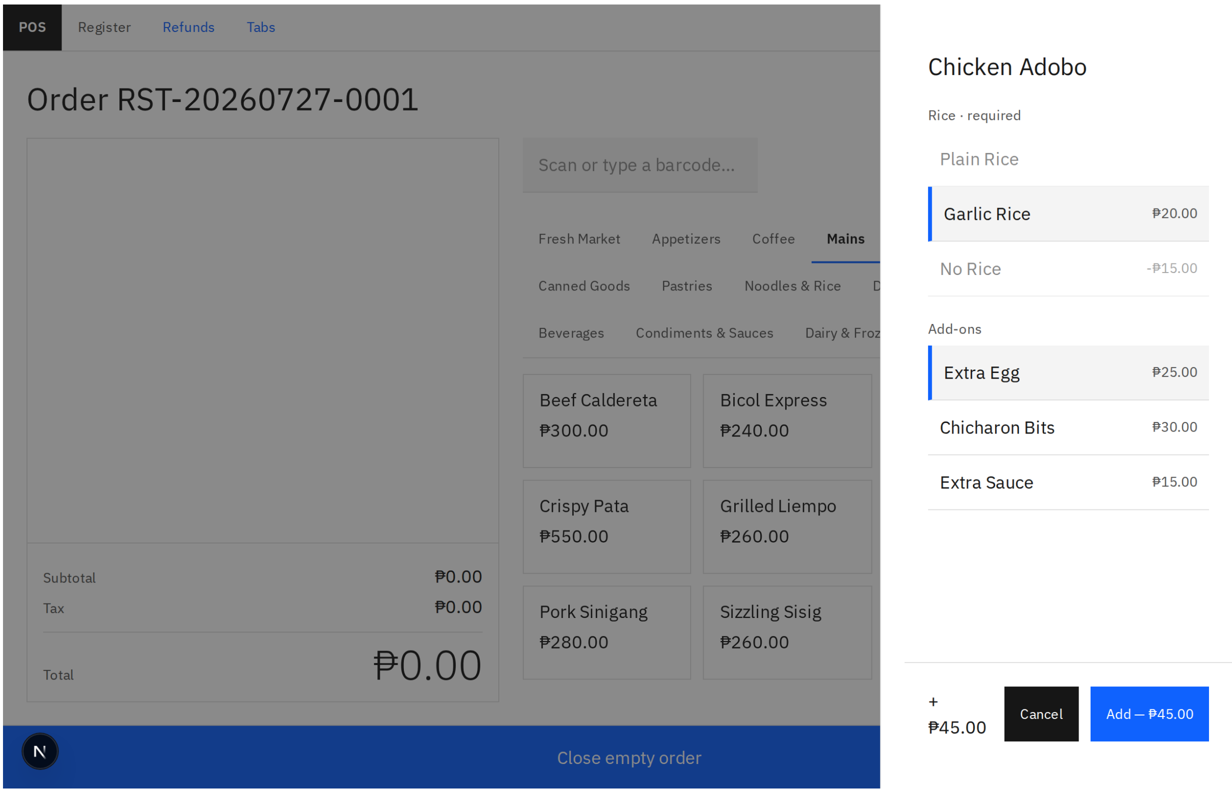
The menu grid and modifiers

Food-mode tills show a category rail and a grid of tiles instead of relying only on the scanner. Tap a category, then tap an item's tile to add it.

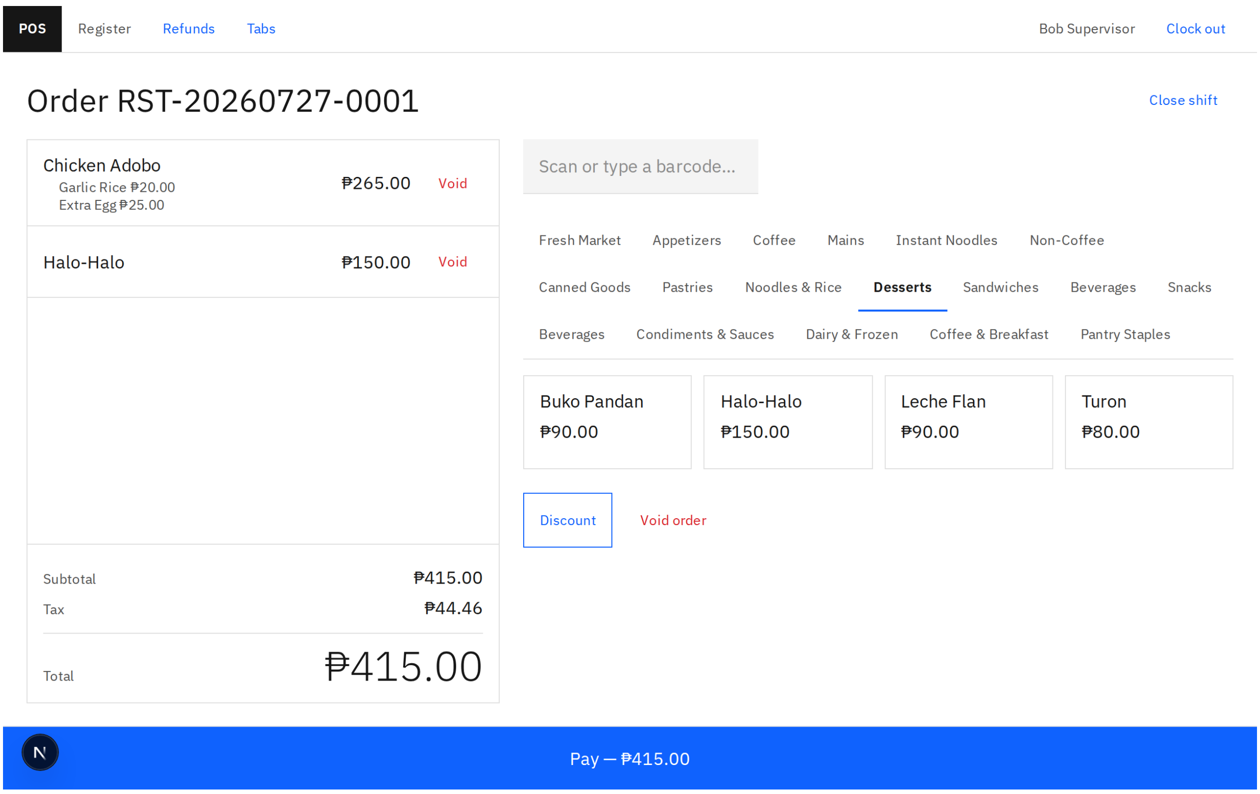


If an item has no modifiers, it goes straight into the cart. If it has any, a sheet pops up named after the item:

1. A group marked **required** needs a pick before **Add** goes live — **Rice** is required on Chicken Adobo, so **Add** stays disabled until one of **Plain Rice**, **Garlic Rice**, or **No Rice** is picked.
2. Tap a choice to select it; tap a repeat-legal one again to add a second (the chip would show ×2).
3. Tap **Add**, or **Cancel** to back out.



A modifier's price is a **signed delta** on top of the item, not always an extra charge — most add to the price (**Garlic Rice** +₱20.00, **Extra Egg** +₱25.00 above), but **No Rice** *subtracts* ₱15.00 instead, because leaving the rice off costs the kitchen less to make. Figure 6.3's picks land in the cart as ₱265.00 for the Chicken Adobo (its own price, plus the two modifiers), right alongside a plain Halo-Halo:



Courses and firing

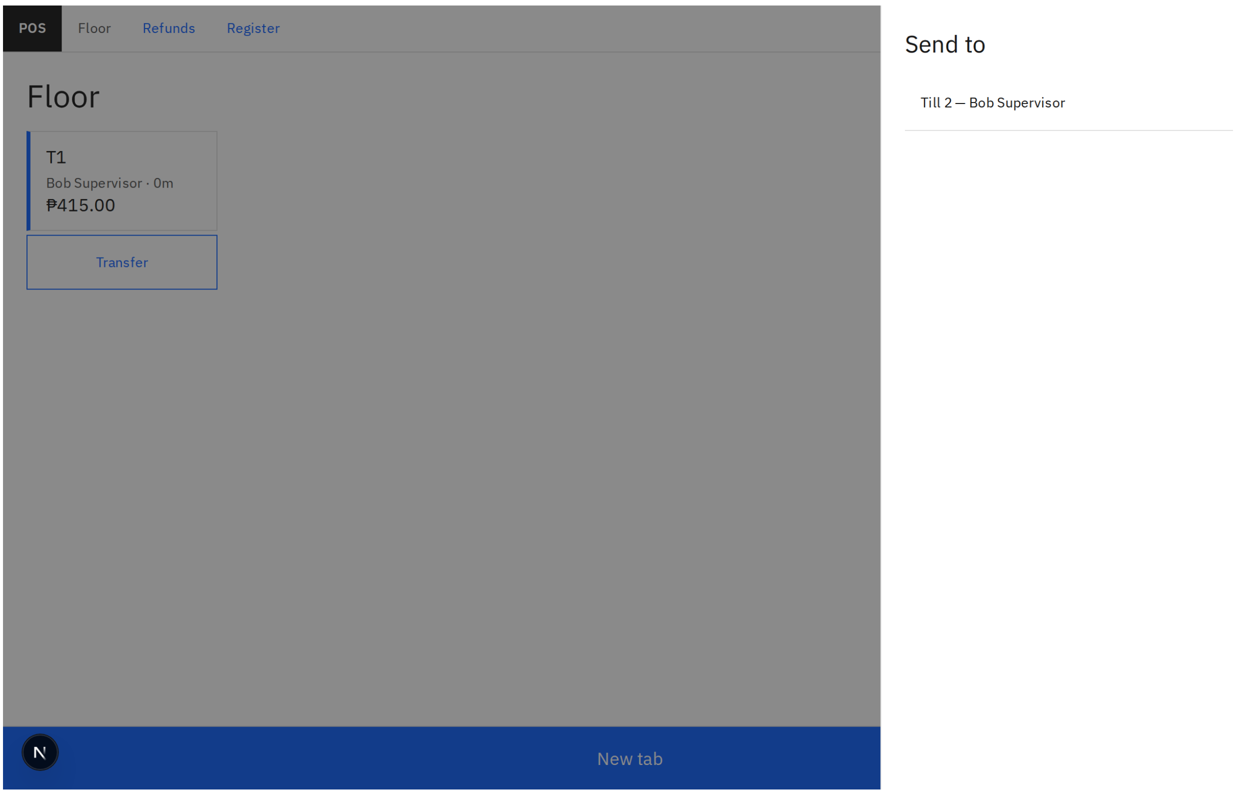
Every line the kitchen tracks carries a chip reading **Pending**, **Cooking**, or **Ready**. Tap it to move the course forward: **Pending** → **Cooking** (fires it to the kitchen) → **Ready** (on the pass) → back to **Pending**. Sending a **Ready** or **Cooking** course back a step, or shrinking its quantity once it's fired, needs the same authority as voiding it outright — downgrading a course the kitchen already started is the same fraud surface as removing it. A cashier's own tap on the chip only ever moves it forward.

> Note: this version's till has no on-screen field to *change* an existing line's quantity at all — only **Void** (remove it entirely) and the prep chip (move it through **Pending** / **Cooking** / **Ready**) are exposed here. A genuine quantity correction on a fired line, or increasing a plain line's count, currently goes through the API directly rather than a till button.

Transfer a tab to another register

On the Floor screen, a tab carries a **Transfer** chip (only if you can transfer, and only when another till is available to receive it).

1. Tap **Transfer** under the tab.
2. Under **Send to**, tap the destination till.

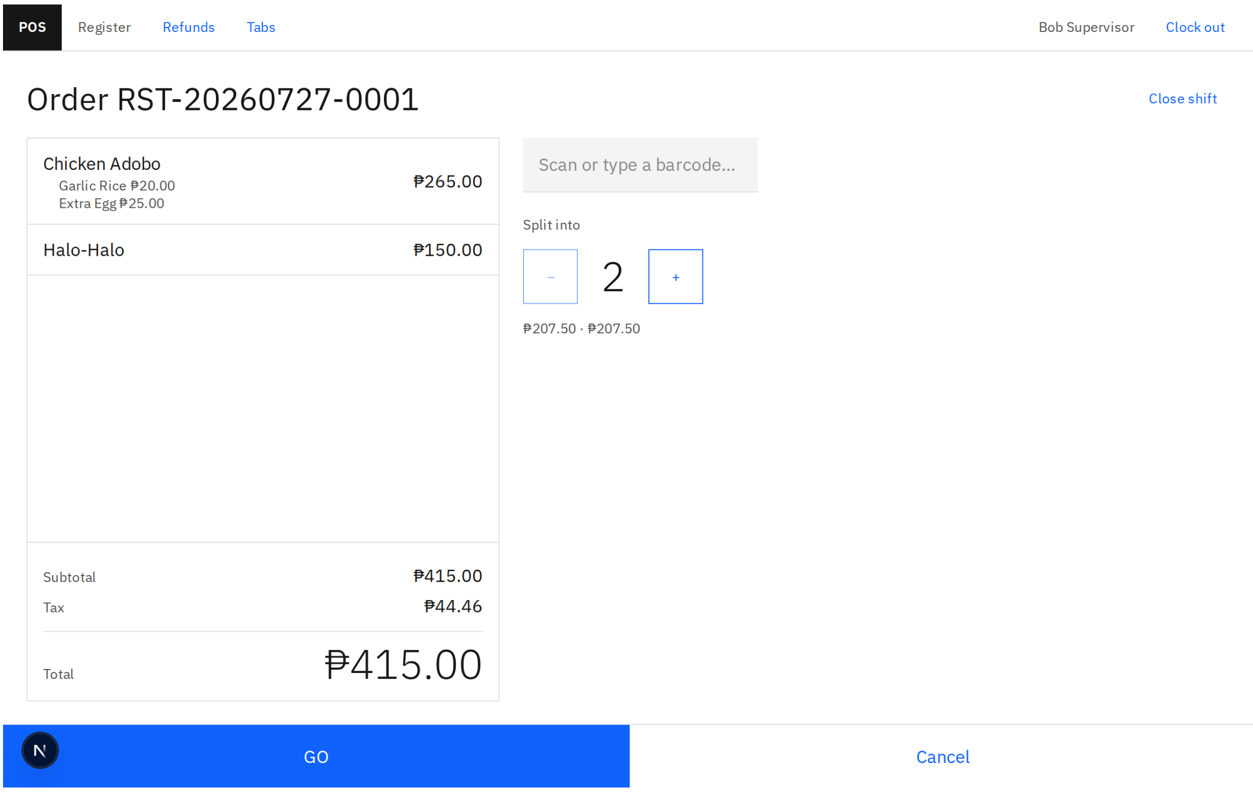


Only a register with an **open shift right now** shows up as a destination — the transfer moves the tab's money onto *that* shift's ledger the instant it lands, so the receiving till's drawer owns the sale from that point on, not the one that rang it up.

Split the check

Once you tap **Pay**, and only on an order that hasn't taken any payment yet, a **Split bill** button appears alongside the tender form.

1. Tap **Split bill**.
2. Use the stepper's **−** / **+** to choose how many checks (2 to 10).



1. Tap **GO**. You're now looking at the first check, with a strip along the top showing **Check 1**, **Check 2**, and so on — take payment on each exactly like any other sale, and the next unpaid one opens automatically as soon as one closes.

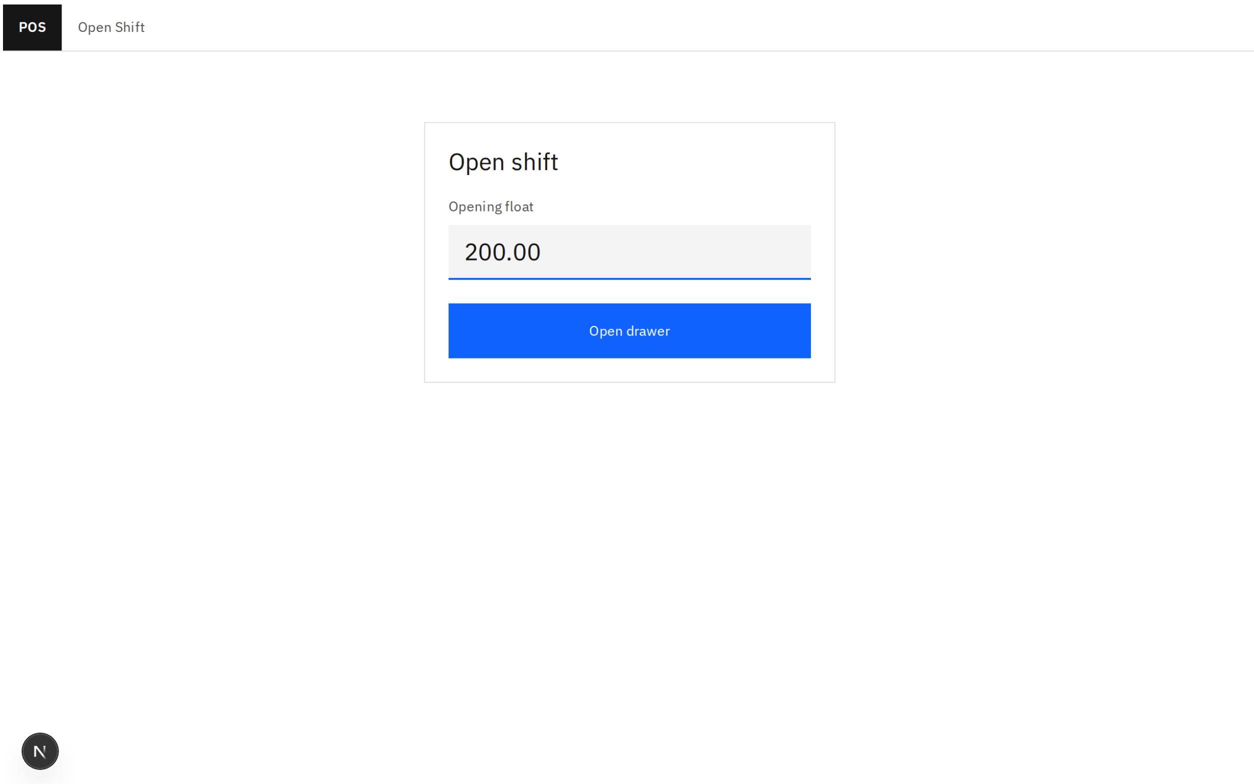
Two even checks, like Figure 6.6's ₱207.50 and ₱207.50, split cleanly. A three-way split rarely does — the totals still always sum back exactly to the original, so a line's quantity on a split check can come out as a fraction (`0.334`, say) rather than a whole number. That's the split dividing everything exactly, not a mistake — the same fractional-quantity field every order line carries, on show here instead of sitting at `1`.

7. Shifts and cash

A till's shift is the drawer's own accountability loop, one level above a single sale: open with a counted float, run the day's sales and cash movements, then close by counting again.

Open with a counted float

If nobody's opened the drawer on this till yet, you land on **Open shift** instead of a sale screen.



Check or edit the **Opening float**, then tap **Open drawer** — you're dropped straight into a new sale. If a shift's already open on this till, you skip this screen entirely.

One thing can refuse an otherwise valid open: **"The business day is closed. Reopen it before opening a shift."** That means a manager has already closed this location's business day for that date (Chapter 15). It isn't a fault with the till, the PIN, or the float, and retyping won't clear it — only an admin can reopen the day, and the drawer opens normally the moment they do.

Cash movements

A payout, a deposit, a paid-in — any cash that moves in or out of the drawer without a sale attached — is a real, audited action, and a reason is required for all of them.

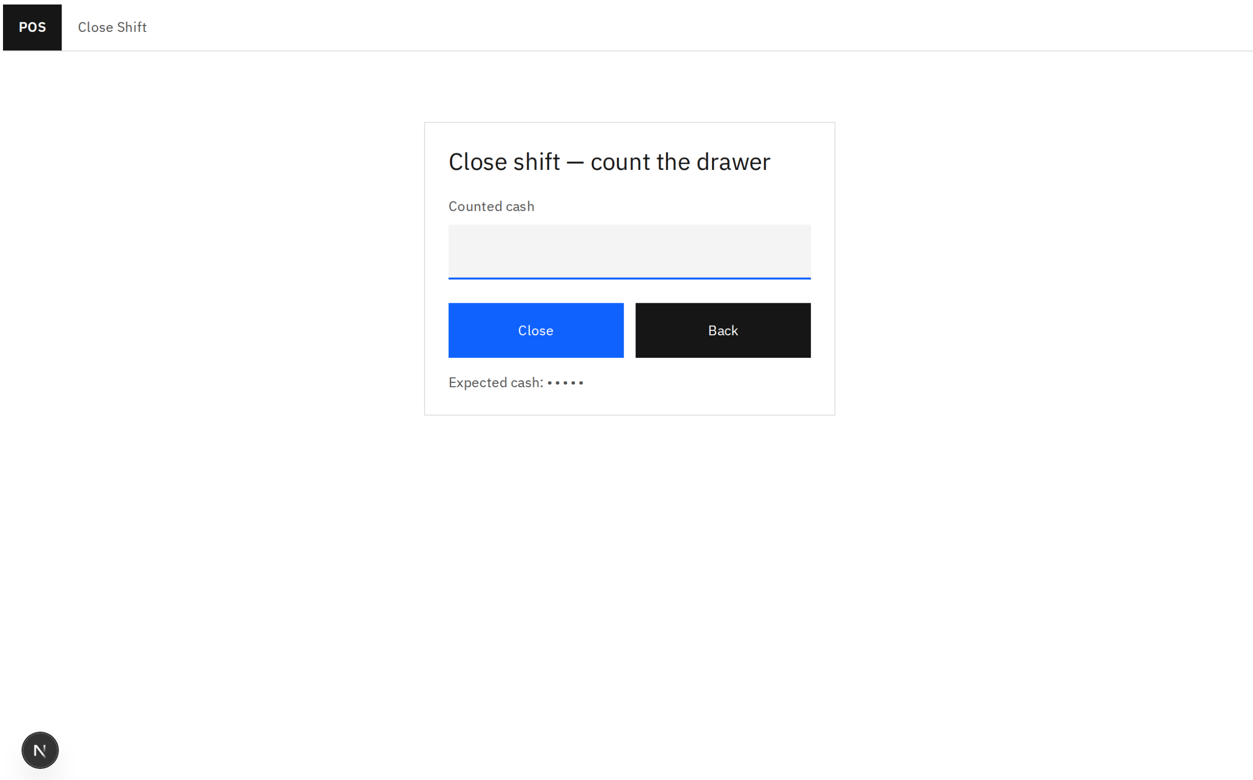
This version's till has no on-screen form to *record* one; the drawer only ever *displays* what's already been recorded against it, in the Z-report below. Recording a movement currently goes through the API directly rather than a till button.

The Z-report — fetched before you close

The Z-report shows up as part of the *same* screen as closing itself, not a separate report you pull up first and close afterward — and that's not just a matter of button order. **Closing a shift revokes every staff session bound to that register the instant it closes**, so the only moment this till can ever show its own Z-report is right there on the close-shift result, before the very session showing it goes dead. There's no "check the Z-report, then close" sequence available on this till at all — the numbers arrive together with the close.

Close and count

1. Tap **Close shift**, in the sale screen's own header row — not the top bar, which just holds your name and **Clock out**.
2. Physically count the drawer *before* looking at anything on screen — the expected figure stays masked (•••••) until after you submit your count.

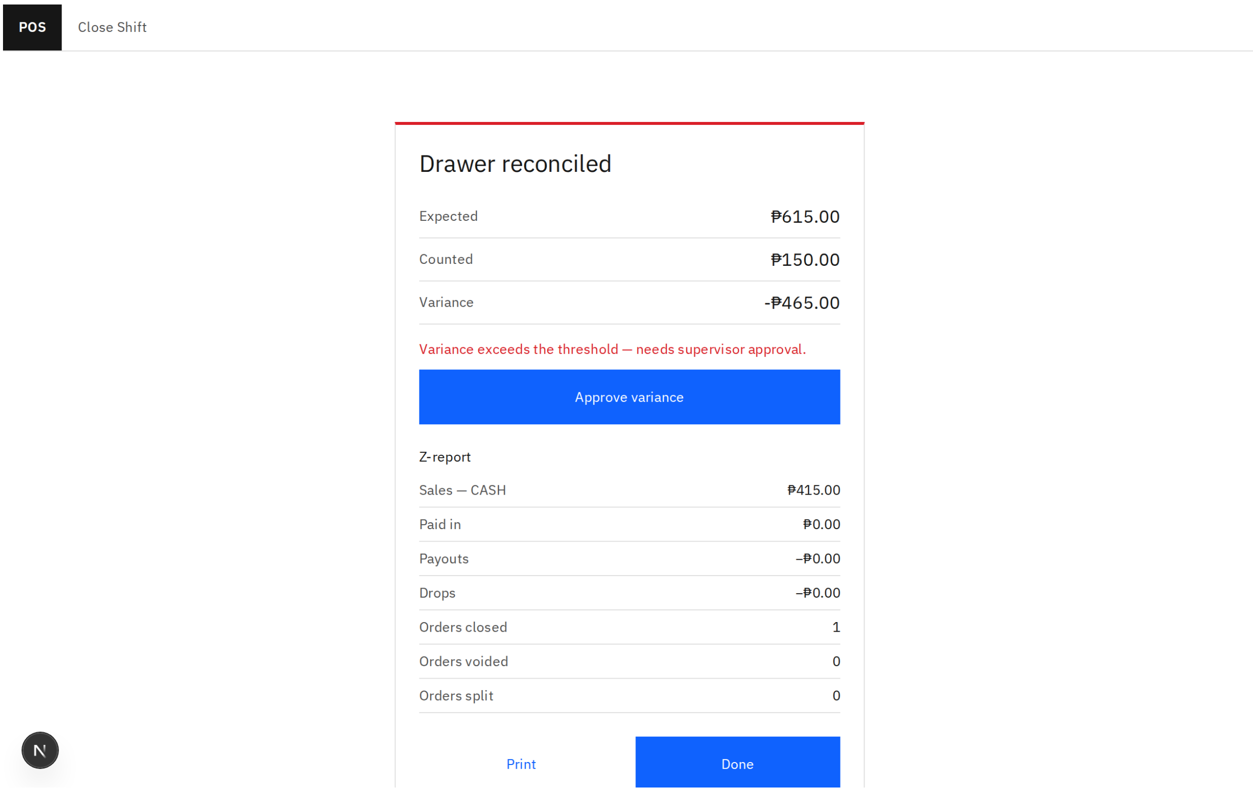


1. Type what you counted into **Counted cash** and tap **Close**.

The masking is deliberate: seeing the expected total first turns counting into just typing the number back, and a real drawer problem never surfaces. Count first, then look.

Variance and approval

The result — **Drawer reconciled** — reveals **Expected**, **Counted**, and **Variance**, plus the Z-report itself: sales and refunds by tender, **Paid in**, **Payouts**, **Drops**, and **Orders closed** / **Orders voided** / **Orders split**.



A variance inside the store's threshold reconciles clean, no extra step. Figure 7.3's -₱465.00 is well outside it, so the screen shows **Variance exceeds the threshold — needs supervisor approval.** with an **Approve variance** button — one that always fails if tapped right here. Closing this shift already revoked this register's own sessions the instant it closed (above), so the very screen offering that button is already running on a dead one; tapping it just signs you back out to **Enter PIN**.

The rule behind it is real, and correctly scoped to the *location*, not one specific terminal: a supervisor approves it from a **different, still-open** register at the same location — never the till that just closed, because that till's sessions are already gone. Approving today means calling the API directly from that other till's session, the same way `scripts/e2e-lunch-service.sh` proves it end to end — no register screen anywhere renders a working **Approve variance** button for a shift other than its own, so this is done outside any till's screen entirely, on purpose, not a bug waiting to be routed around.

Tap **Print** for a paper copy of the Z-report, then **Done**. Closing signs you out of this till completely — you land back at **Enter PIN**.

Finding what needs approval: the Variances queue

With more than one register, "which shift needs a variance approved" isn't something any single till can tell you — you'd have to sign in at each one in turn to find out. The back office's **Variances** section (Chapter 8) answers that without the walk: every closed shift, at every location you cover, whose variance is still over threshold and unsigned, newest close first — register, who opened it, when it closed, the expected, counted, and variance figures, and the threshold it was measured against (each location can set its own, so the same variance can qualify at one store and not another).

It's a list, not a shortcut past the rule above — there's no **Approve** button in the back office. The page says so right above the table: this is where to find which shifts need sign-off, approval is scoped to the location and never to the till that closed (closing already revoked that till's own sessions), and today that means calling the API directly from another still-open register's session at the same location — no till screen anywhere offers an Approve button for a shift other than its own. Use the queue to see which shift and location need attention, then approve it exactly as the section above describes: an API call from that other, still-open till's session, not a screen at any till.

8. The back office

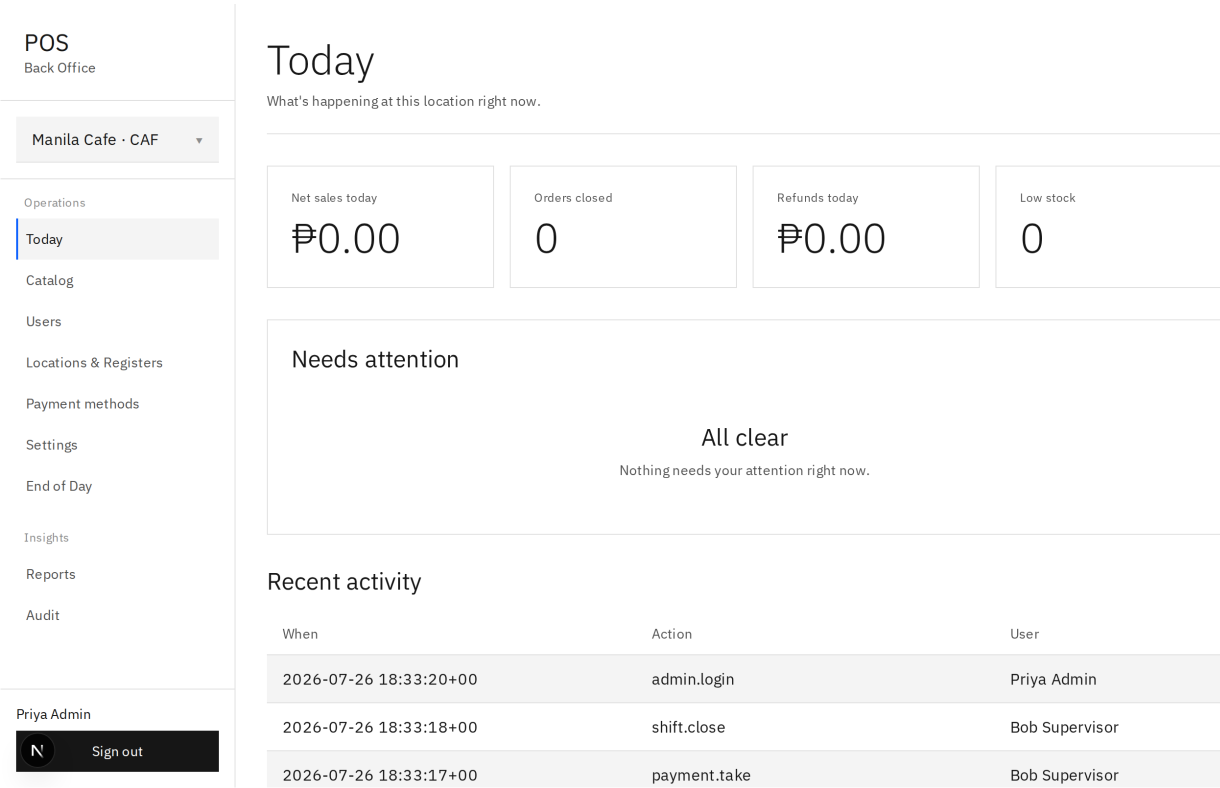
Layout

Everything in the back office sits behind one sign-in. Once you're in, a rail down the left holds every section your permissions unlock, under two headings — **Operations**: **Today**, **Catalog**, **Users**, **Locations & Registers**, **Variances**, **Payment methods**, **Settings**, **End of Day**; **Insights**: **Reports**, **Audit**. **Today** always shows; every other section only appears if you hold a permission that unlocks it — an admin sees all ten, a manager granted only sales reporting sees **Today** and **Reports**. A **location switcher** sits above the rail, and your name plus a **Sign out** button sit at the bottom.

Today, **Reports**, and the **Stock** report all read whichever location the switcher is set to — there's no separate location picker on each of those screens. Change the location once, at the top, and every screen that cares about it follows along.

Today

Today is what you land on right after signing in — a glance at whichever location the switcher is set to, right now, with nothing to configure.



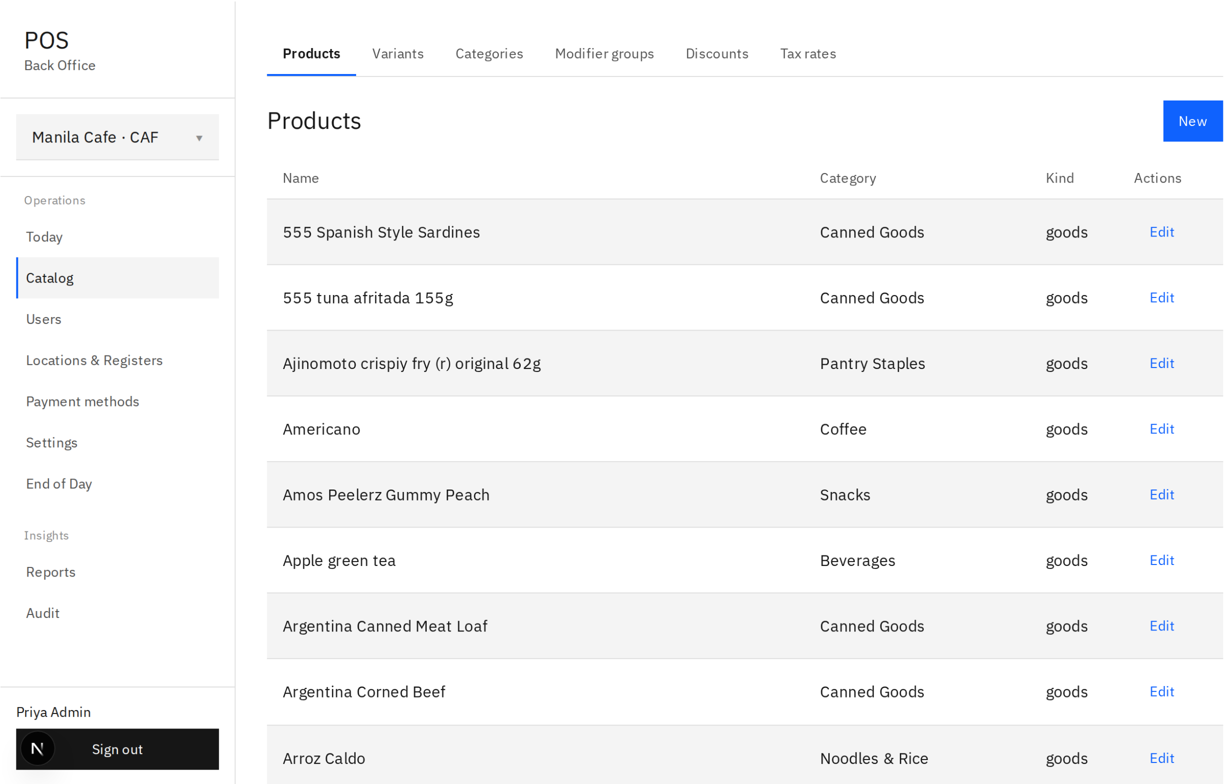
The screen shows:

- A row of four figures — **Net sales today**, **Orders closed**, **Refunds today**, and **Low stock** — the same numbers the Sales and Stock reports show, just for today and this location.
- **Needs attention** — a table of every low-stock variant and every inactive register at this location. Nothing to flag shows **"All clear"** instead of an empty table, exactly as it does in the figure above.
- **Recent activity** — the first page of the audit log (When, Action, User), trimmed to a glance.

Every number on **Today** already exists somewhere else in the back office — gathered onto one screen rather than computed specially. If a figure here ever looks off, the matching report (Reports, Stock, Locations & Registers, Audit) is where to double-check it.

9. Catalog

Catalog is where the menu lives — categories, products and their variants, modifier groups, discounts, and tax rates, each on its own tab.



Products and variants

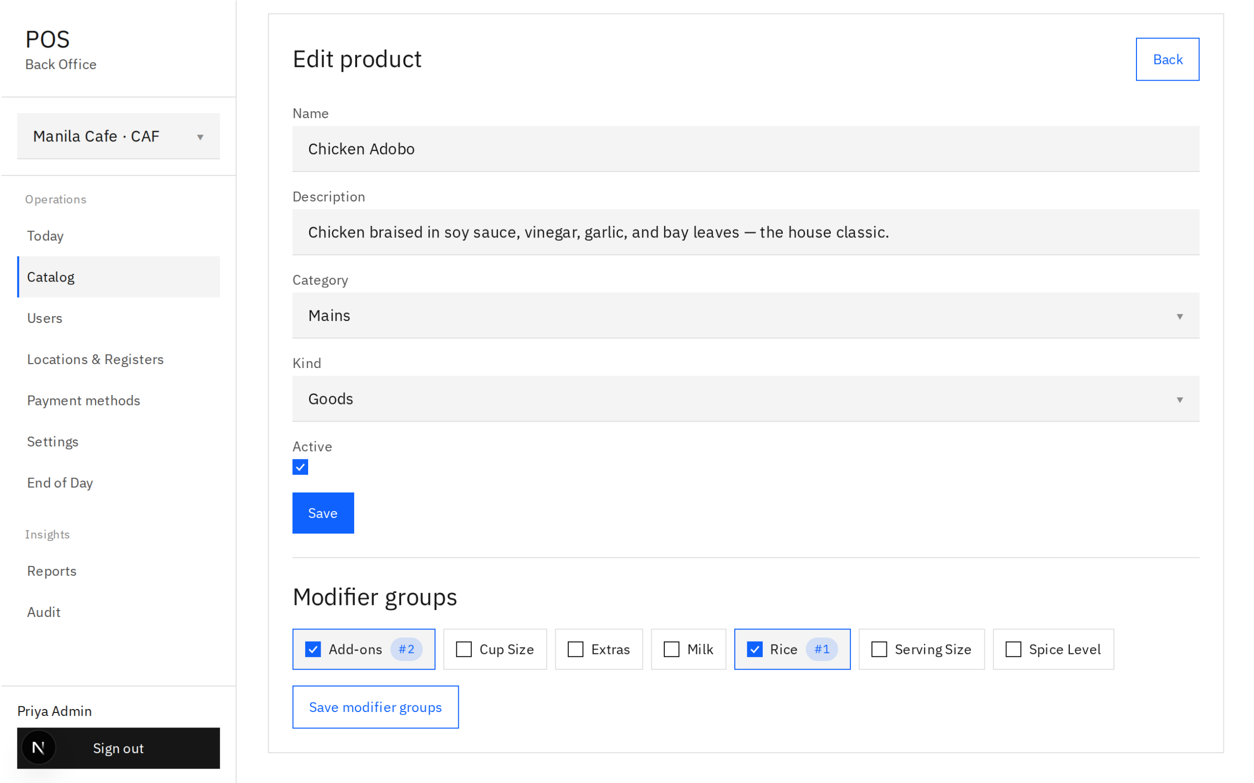
A **product** is the thing on the menu — a name, a description, a category. A **variant** is what's actually sold under that product: its own SKU, barcode, price, cost, and tax rate. Every product needs at least one variant before it can be rung up at all.

1. In **Catalog**, tap **Products**, then **New**.
2. Fill in **Name**, **Description**, **Category**, and **Kind** (**Goods** or **Service**).
3. Tap **Save**.
4. Tap **Variants**, then **New**.
5. Pick the product, then fill in **Name**, **SKU**, **Barcode** (optional), **Price**, **Cost** (optional), **Tax rate**, and **Track inventory**.
6. Tap **Save**.

Prices and costs are typed as pesos (e.g. `120.00`) but travel to the server as cents — a blank or unparseable price is refused before it's ever sent.

Modifier groups

To attach a modifier group (a coffee's milk, a dish's extras) to a product, open the product again — a **Modifier groups** panel at the bottom lists every group as a checkbox.



The figure above shows **Chicken Adobo** with **Rice** ticked first (shown as **#1**) and **Add-ons** ticked second (**#2**) — the order you tick groups in is recorded against each checkbox. Tapping **Save modifier groups** is a separate action from the product's own **Save** button above it, and it's a full replace of the product's attached groups, not an add-one/remove-one — so an untouched box you meant to keep still has to be ticked before you save.

Categories and modifier groups themselves have no archive toggle — only products, their variants, and the modifiers inside a group, do. A group with **Min select** at 1 or more shows up as *required* at the till: the cashier can't confirm an item until every required group has a pick.

Discounts and tax rates

Discounts (its own tab): **Name**, **Kind** (**Percent** or **Fixed amount**), the matching value, **Scope** (**Order** or **Line**), **Requires supervisor**, **Active**.

Tax rates (its own tab): **Name**, **Rate (%)** — typed as a percent (e.g. `12.00`), stored and computed on the server as an exact fraction so the math never drifts.

Archive, never delete

There is no delete button anywhere in the back office. Unchecking **Active** and tapping **Save** on a product, variant, discount, or tax rate **archives** it instead — it leaves the register's menu, but every past order line, receipt, and report that points at it keeps working, forever. A confirm dialog asks first — **"Archive *Name*? It leaves the register catalog but stays in history."** — and an archived row shows greyed with an **ARCHIVED** badge and an **Unarchive** button.

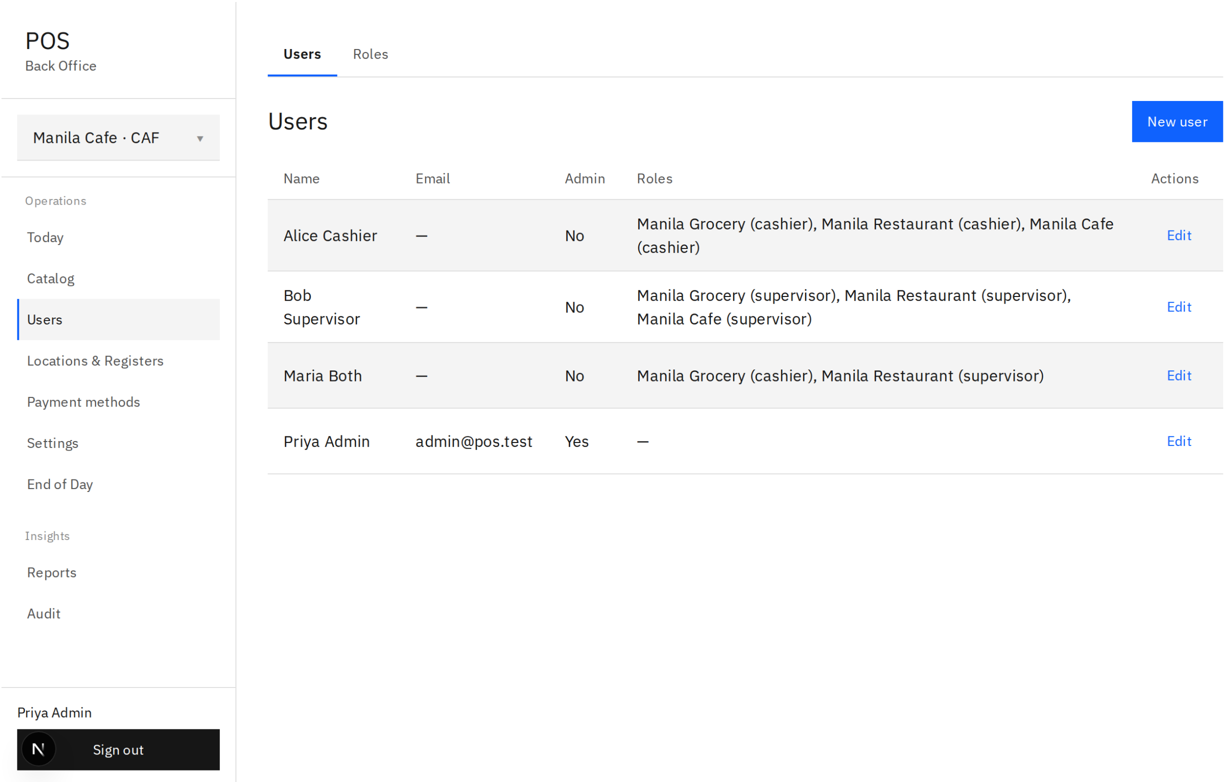
Common issues

Symptom	Cause	Fix
I archived an item and it's still on the till	The register's menu caches the catalog for 5 minutes	Normal — wait it out, or scan the barcode instead (a scan always hits the server)
A receipt from last month shows the old price after I repriced the variant	Every order line snapshots its price when it's added — receipts never look up the live catalog	Expected behavior, not a bug — see Chapter 2 for how this is proven
A new product has no Modifier groups panel	The panel only appears once the product has been saved once (so it has an id)	Save the product first, then open it again

10. Users and roles

Hire a cashier or a supervisor

1. In **Users**, tap **New user**.
2. Fill in **Name**, and either an **Email**, a **PIN**, or both — one of the two is required for a new hire.
3. Leave **Admin** unchecked for regular staff.
4. Under **Roles**, pick a location in **Add location**, pick a role in **Add role**, then tap **Add**. Repeat for every location this person works at.
5. Tap **Save**.



The figure above shows the shape this takes in practice: **Alice Cashier** holds the cashier role at all three locations, **Bob Supervisor** holds supervisor at all three, **Maria Both** is a cashier at Manila Grocery and a supervisor at Manila Restaurant, and **Priya Admin** — the only row with an email and no location roles at all — is a full admin instead.

Saving the **Roles** table writes the *complete* set of that person's role assignments in one action, the same way saving modifier groups does in Chapter 9 — tap **Remove** next to a row to drop a single location assignment before you save, rather than expecting to edit it after the fact.

Cashier and **Supervisor** ship as the two starting roles, but roles are no longer fixed. A user with the **Manage roles** permission (or **Admin**) sees a **Roles** tab next to **Users**: every role's name, its permission checklist, and how many people hold it. The two built-in roles keep their name (so nothing that assumes **Cashier**/**Supervisor** exist ever breaks) but their permissions can still be edited; a role added from this tab can be renamed or removed freely, except removal is refused while anyone still holds it.

Give someone back-office access

Back-office access is no longer all-or-nothing. Checking **Admin** still gives full access to every section, at every location. Short of that, holding **any** back-office permission — through a role, or a one-off grant added directly on the user's own record — is enough on its own: that person signs in with their email and password and the sidebar shows exactly the sections their permissions unlock, nothing more.

Deactivate a leaver

Open their user record, uncheck **Active**, and tap **Save** — you'll be asked to confirm: **"Deactivate *Name*? They keep their history but can no longer sign in."** Their name still shows correctly on every past order, audit entry, and report they touched; deactivating only blocks a future sign-in. A **Reactivate** button brings them back later if needed.

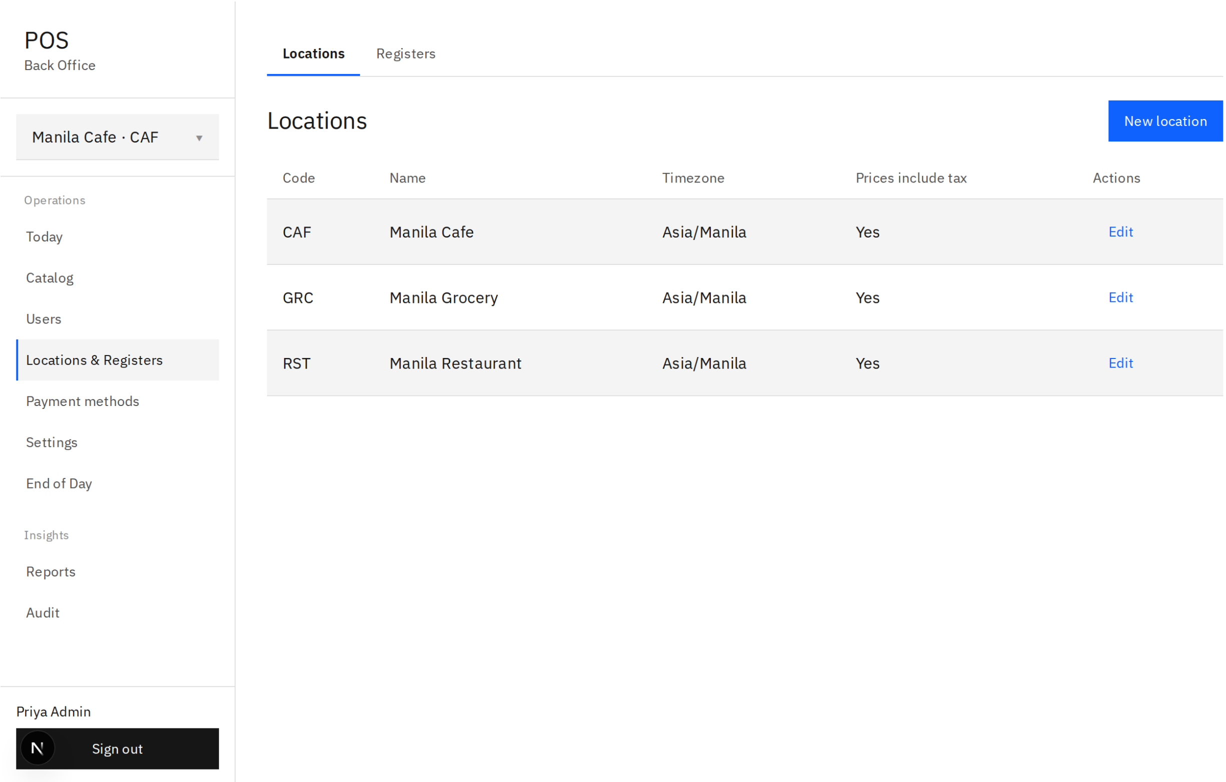
The self-lockout guard

You cannot uncheck your **own Admin** box, and you cannot uncheck your **own Active** box. Either one is refused outright, because there's exactly one admin tier and no fallback above it — if you could lock yourself out, there might be no one left who could undo it. Have a second admin make the change if one genuinely needs to leave.

11. Locations and registers

Location settings

Locations & Registers → **Locations** tab lists every store: **Code**, **Name**, **Timezone**, **Prices include tax**, with an **Edit** action.



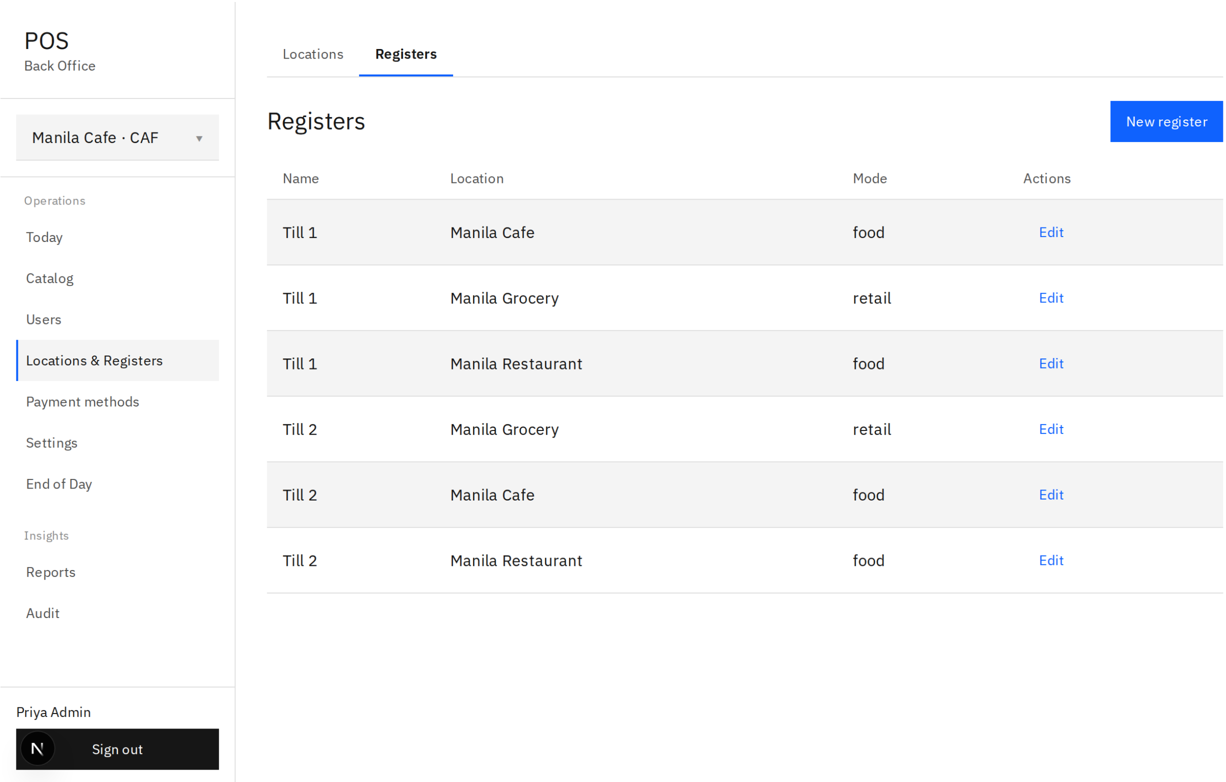
All three of the demo locations shown above — **Manila Cafe**, **Manila Grocery**, **Manila Restaurant** — run on the `Asia/Manila` timezone with **Prices include tax** set to **Yes**. Editing a location lets you change its **Name**, **Code**, **Timezone** (checked against the IANA list), **Prices include tax**, **Receipt header**, **Receipt footer**, **Variance approval threshold**, and **Low stock threshold**. Flipping **Prices include tax** only applies to future orders — an order already open keeps the pricing basis it started with.

The two threshold fields are optional — leave either blank and this location follows the store-wide default. Set one to override just this location: a busier store might tolerate a larger drawer variance before a close needs a supervisor's approval, or want its low-stock warning to fire sooner. Blank means "use the default," not zero, so clearing a field you'd previously set puts the location back on the default rather than turning the check off.

A location can be deactivated the same way as everything else in the back office — uncheck **Active**, confirm, and staff can no longer sign in there, though its history stays intact.

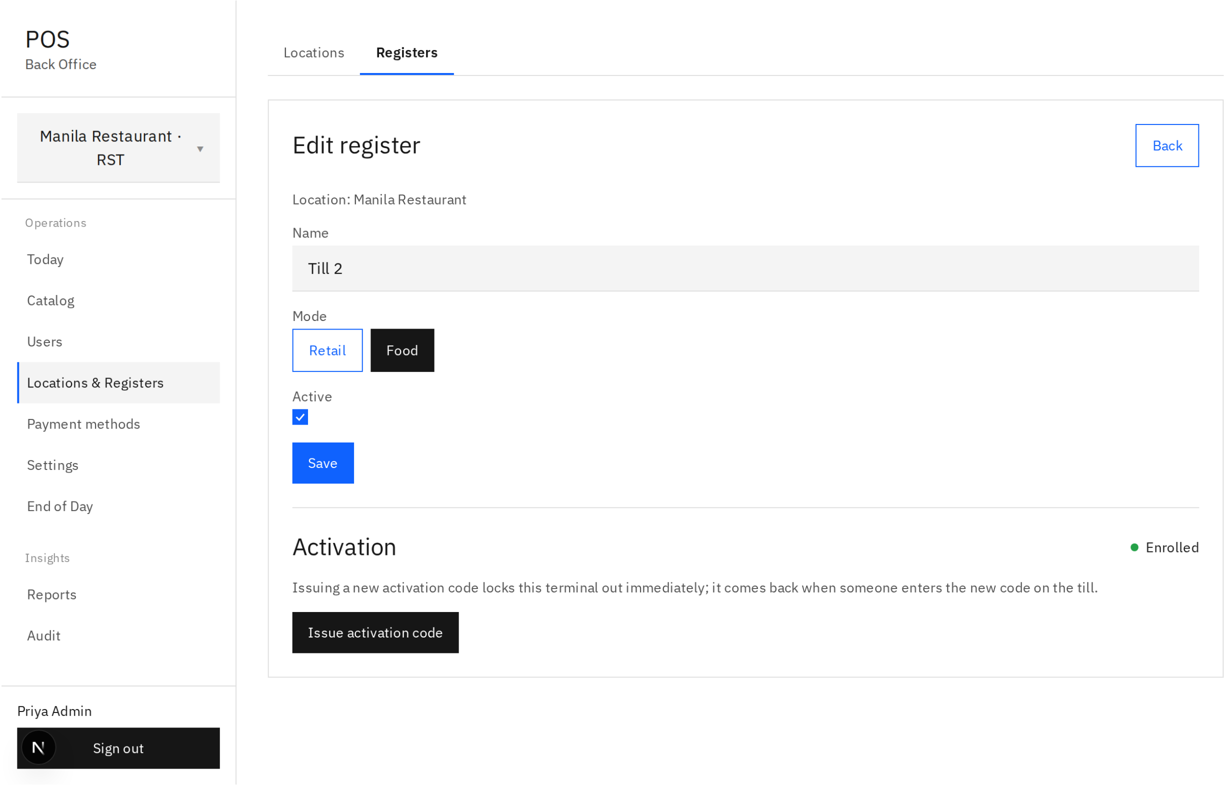
Register mode

Locations & Registers → **Registers** tab lists every till across every location, with its **Mode**.



`Till 1` at Manila Grocery runs **retail** — the scan-and-cart screen; every other till in the figure above runs **food** — the floor-and-menu-grid screen. Flipping a register's mode is a two-tap edit:

1. **Locations & Registers** → **Registers** → **Edit** on the till.
2. Under **Mode**, tap **Retail** or **Food**.
3. Tap **Save**.



There's no in-between — it's one mode or the other for the whole register, and it decides which screen that till's app shows the next time someone signs in there. A register can be deactivated the same way as a location; **Reactivate** brings it back.

On-screen keyboard

Some tills have no physical keyboard attached — a sealed all-in-one terminal, or a tablet in a stand. **On-screen keyboard** puts a touch keyboard on that till, docked at the bottom of the screen whenever a cashier taps into a field: cash tendered, PINs, a barcode typed after a failed scan, table refs, card references, and the reasons typed for a void, discount, or refund.

1. **Locations & Registers** → **Registers** → **Edit** on the till.
2. Check **On-screen keyboard**.
3. Tap **Save**.

This is a per-till setting, not a per-location one — a store commonly mixes hardware, and a sealed counter terminal beside an ordinary PC enrolled as a second till can have the toggle on for one and off for the other. It defaults off, since most tills already have a keyboard attached.

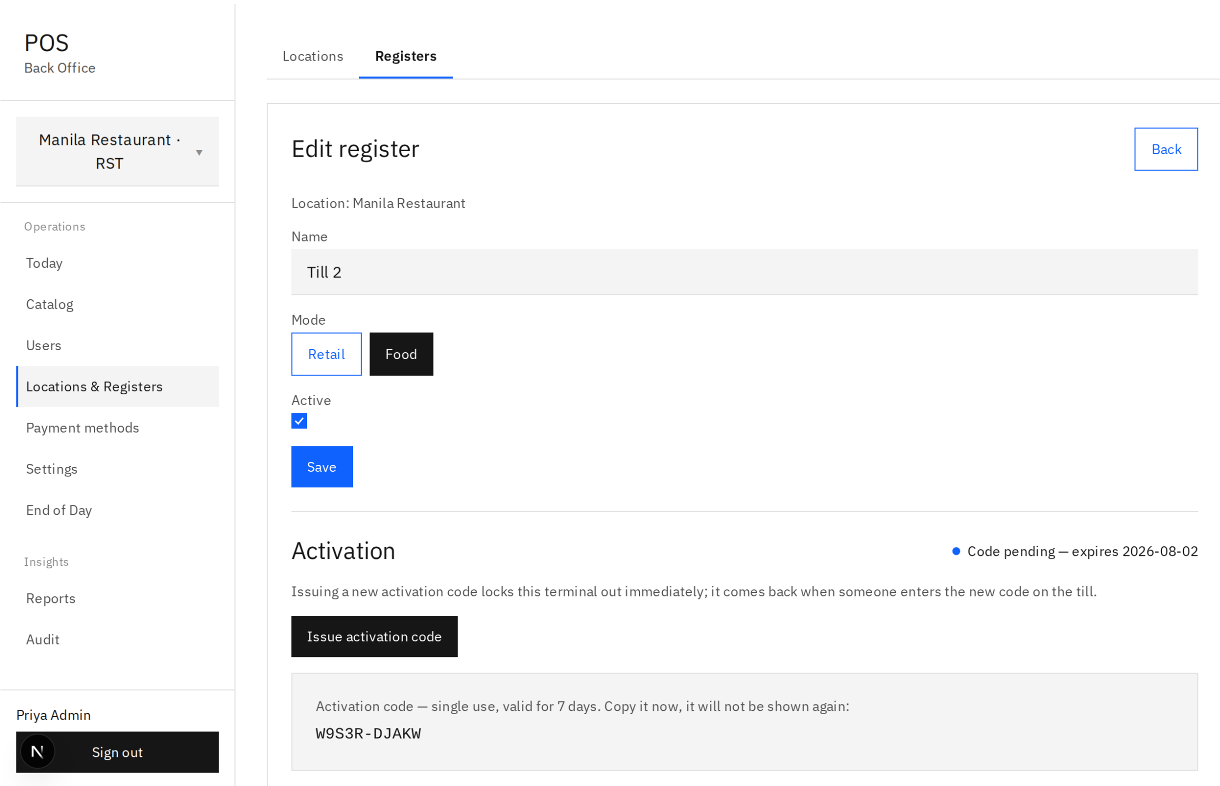
One thing it doesn't cover: the very first activation of a keyboard-less terminal (Chapter 3, and the **Issue an activation code** section below). At that point the till isn't yet a known register the app can read the setting from, so typing the activation code in still needs a physical keyboard attached, just for that one step — or the code typed on a phone and pasted in. Once the till is activated, the toggle above takes over.

Issue an activation code

A till proves itself with a device token, but a manager never handles that token directly — the back office only ever deals in **activation codes**, which the till itself exchanges for its device token. Every register's editor shows an **Activation** status pill: **Enrolled**, **Code pending — expires *date***, **Code expired**, or **Not enrolled** — Figure 11.3 above shows **Enrolled**.

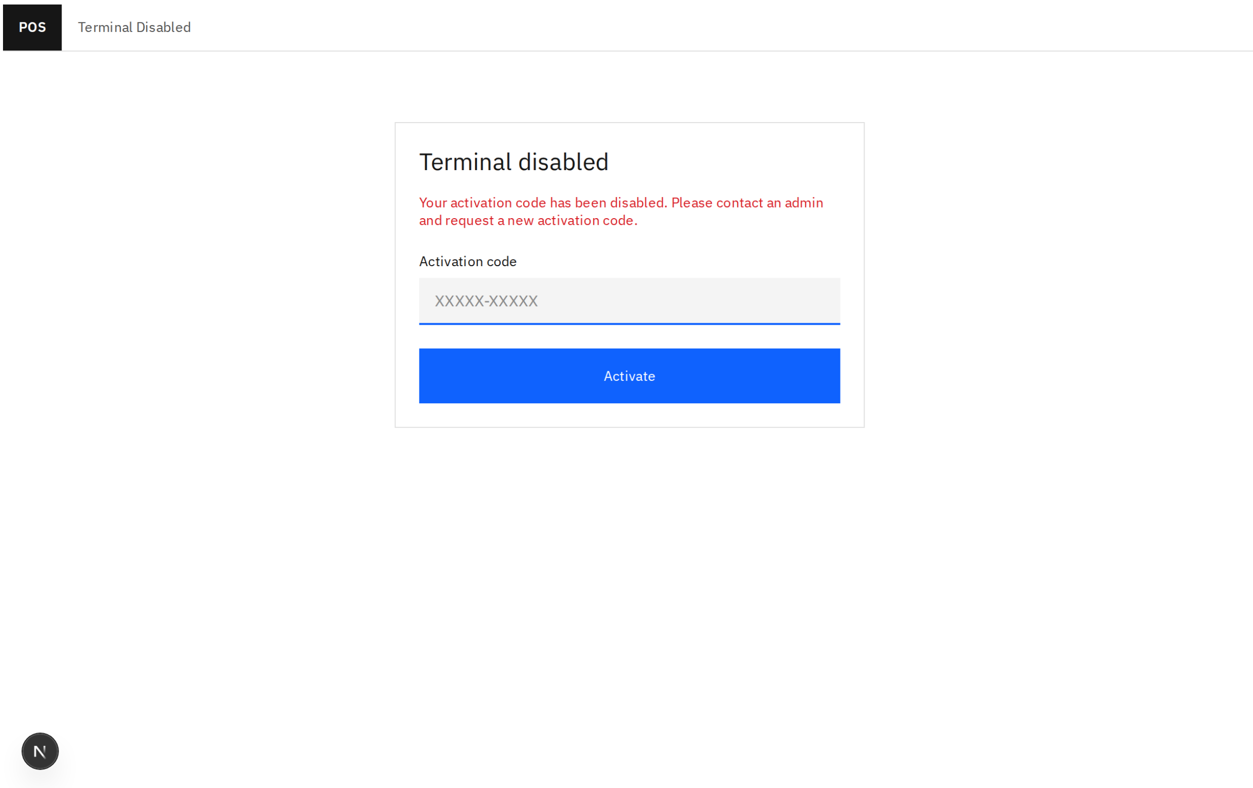
This is also how a brand-new register goes online for the first time — there is no separate "first code" screen. **New register** → fill in **Location**, **Name**, **Mode** → **Save**, then reopen that same till with **Edit** and follow the steps below, exactly as they apply to replacing a lost or stolen terminal's code.

1. **Locations & Registers** → **Registers** → **Edit** on the till.
2. Under **Activation**, tap **Issue activation code**.
3. Confirm: **"Issue a new activation code for *Name*? The current till goes dark immediately."**
4. The new code appears once, in a copy-me plate.
5. Type the code into the terminal's own **Activate this terminal** screen (Chapter 3).



The figure above shows the plate exactly as it appears: **"Activation code — single use, valid for 7 days. Copy it now, it will not be shown again:"** followed by the code itself, and the status pill already reading **"Code pending — expires 2026-07-29"**. Copy it now — closing this screen and it's gone for good; it's never written anywhere the back office can show again.

Issuing a code revokes the register's current device token *and* every staff session bound to it, in the same transaction that stores the new code — there's no window where a lost terminal and its replacement are both live. The old till drops back to a lockout screen the instant it next tries to talk to the server:



The message reads exactly what it says: **"Your activation code has been disabled. Please contact an admin and request a new activation code."**, with the activation-code entry form right below it — so anyone still holding the old terminal is locked out immediately, not eventually, and can get back in the moment the new code is typed in. A code that expires unused (7 days) shows **Code expired** instead — issue a fresh one the same way.

12. Payment methods

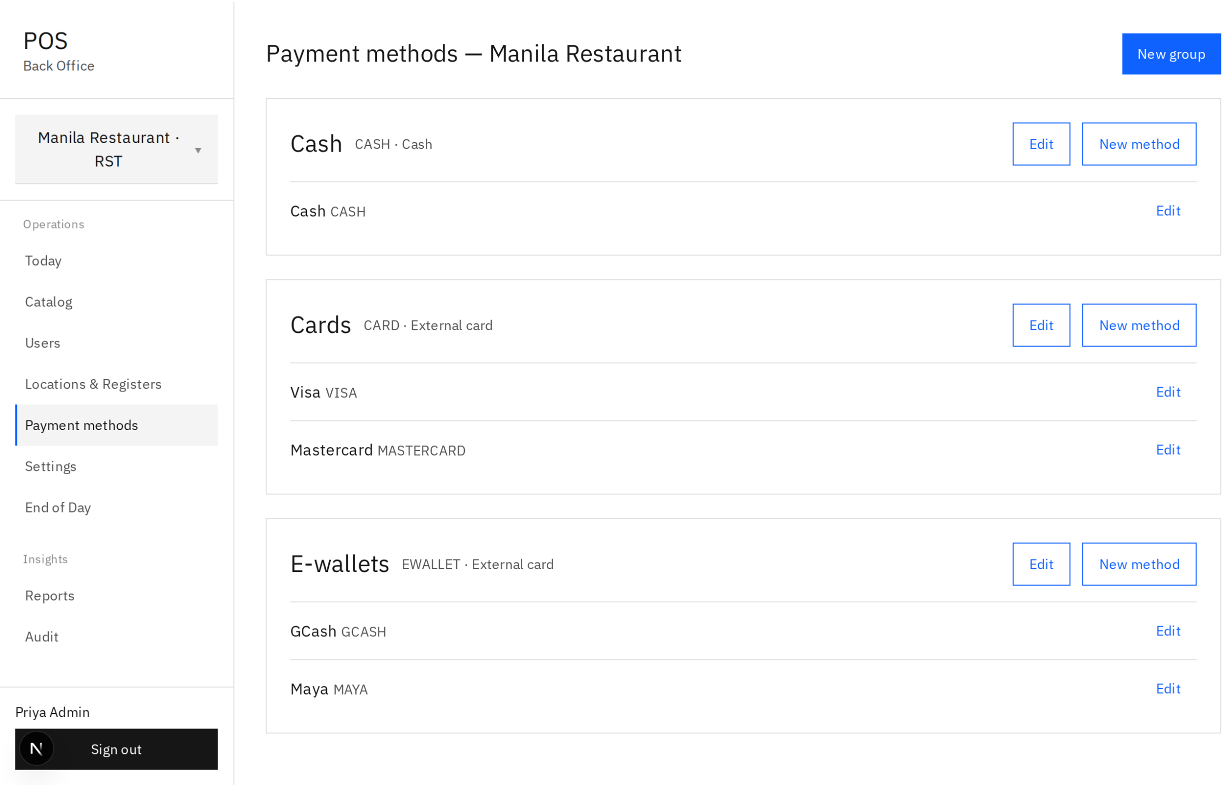
Every location decides for itself what it accepts. A **payment method group** is a kind of tender — Cash, Cards, E-wallets — and a **payment method** is a name inside it: Visa and Mastercard under Cards, GCash and Maya under E-wallets.

The split matters because the *group* is what decides behaviour. A cash group opens the drawer and works out change. Every other group records what another machine already did, and money taken on it can't be refunded through this till — the money never came through here.

Add a method

Pick the location in the sidebar first: these are that location's tenders, and two locations can use the same codes for different things.

New group asks for a code (like `EWALLET`), a name customers would recognise, and which of the two behaviours it has. **New method** inside a group asks only for a code and a name — it inherits the group's behaviour, which is the point.



Codes are letters, digits and underscores, and they're what your reports are grouped by, so pick them once and keep them.

What you can change later, and what you can't

Names and order can change whenever you like — they're what staff read.

Codes can't, and neither can a method's group. A code is what the till sends and what your reports are keyed on; a group is the behaviour. Changing either after money has moved would quietly rewrite history. If one is wrong, archive it and add the right one.

Archive, never delete

Archiving a method takes it off the till and leaves every past sale intact. Archiving a whole *group* takes all of its methods off the till at once — one switch for "we've stopped taking cards" — and reactivating it brings back exactly the ones that were live before.

A location with nothing active can't take payment at all. That's allowed (a card-only kiosk is a real thing), so nothing stops you — the till just says so.

Reading it back

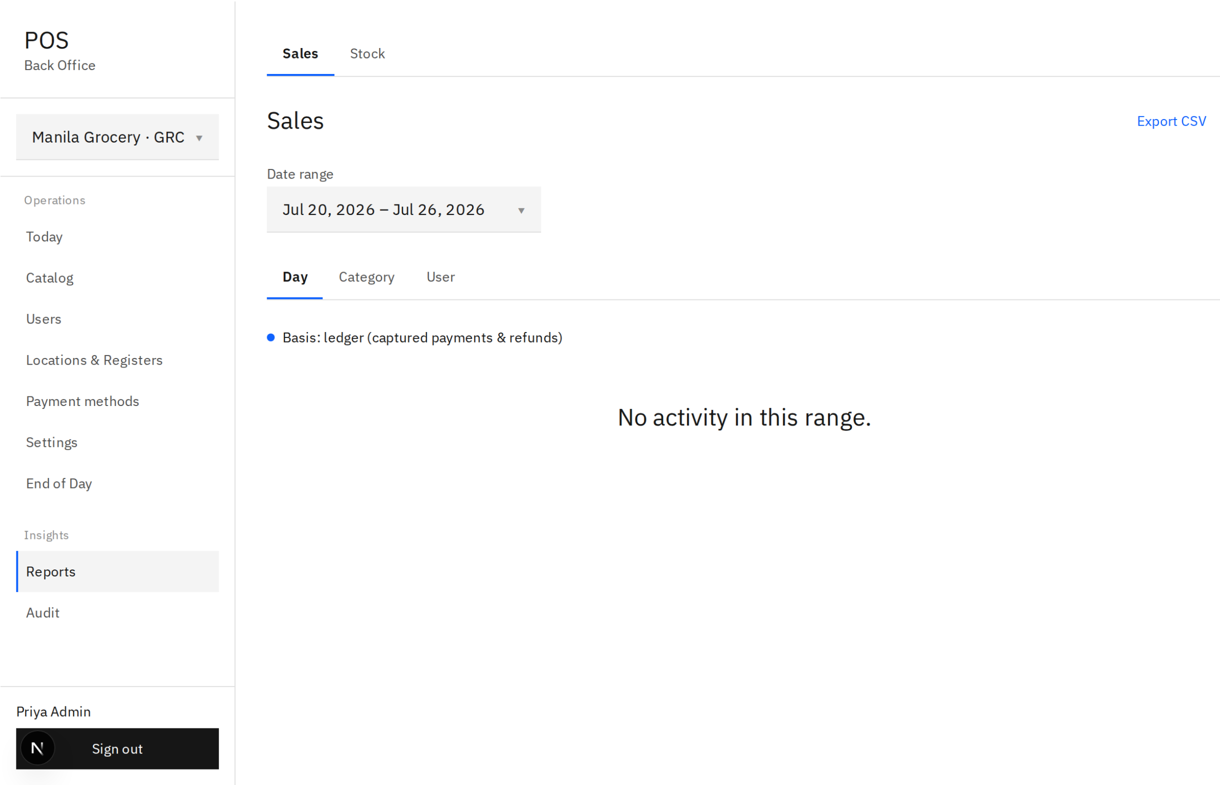
The Z-report at the till (Chapter 7) breaks sales and refunds down by individual payment method — every method gets its own line, so Visa and GCash always show up separately from each other and from Cash. When the location has more than one group active, it also rolls those same lines up by group — CARD and EWALLET both drive `external_card`, so the group rollup is the only place a supervisor counting the drawer sees GCash apart from Visa. A location with only one group active skips the rollup; it would just repeat the method lines above.

The back office's **Reports → Sales** (Chapter 13) also offers a **Payment method** grouping, alongside **Day**, **Category**, and **User**. "How much came in on GCash last month" is answered directly by that one report screen, picking **Payment method** and the range in question — not by adding up a figure across a range of Z-reports.

13. Reports

Sales

Reports → Sales tab: pick a **From/To** date range, then a group-by tab — **Day**, **Category**, **User**, or **Payment method**. The location comes from the sidebar's location switcher.



Day and **User** are **ledger-basis** — summed from actual payments and refunds that moved money, with columns **Orders closed**, **Gross**, **Refunds**, **Net**. The figure above shows exactly one order closed on 2026-07-22 at Manila Grocery: **₱405.00** gross, **₱0.00** refunded, **₱405.00** net.

Category is **line-basis** instead — summed from order lines, joined to *current* category names, with columns **Qty sold** and **Line total**.

Payment method is ledger-basis too, one row per method that had activity in the range, with columns **Method**, **Code**, **Group**, **Gross**, **Refunds**, **Net** — the method's name and its group are shown side by side with the code the ledger is actually keyed on, so "how much came in on GCash last month" is one row, not an addition exercise.

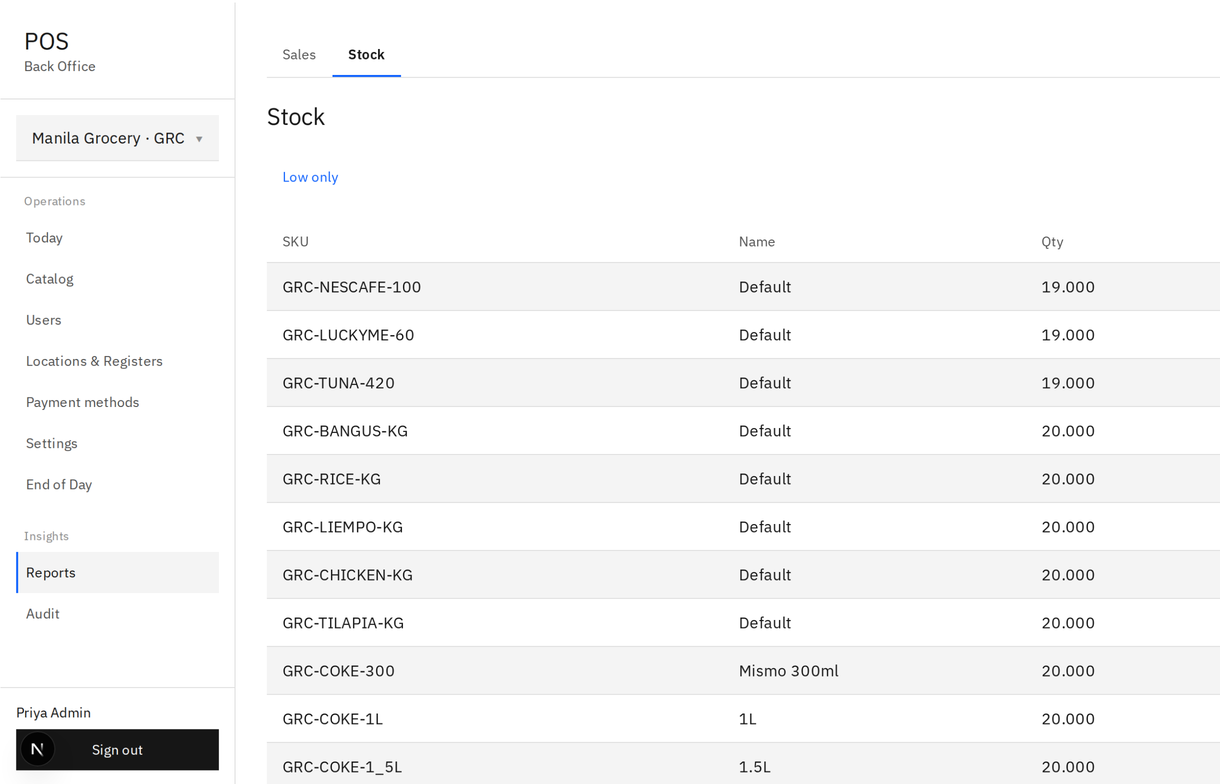
These bases are **not required to reconcile** with each other, and that's by design, not a bug to chase down. A payment covers a whole order at once; a category breakdown has to attribute individual lines. They're answering different questions —

"how much cash and card came in" versus "what sold" — so don't expect the Day total and the sum of the Category totals to match to the cent.

Tap **Export CSV** on the **Sales** tab to download the report exactly as displayed, with money figures as plain decimal strings rather than currency-formatted text.

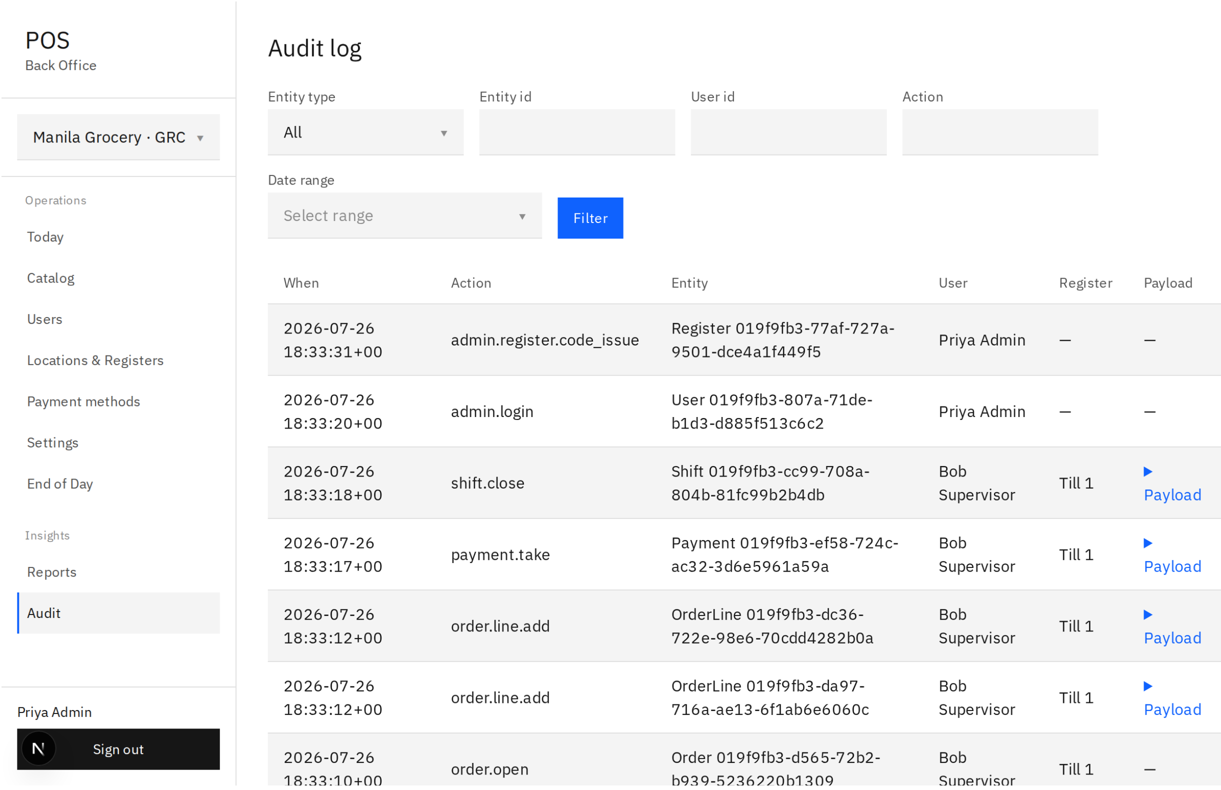
Stock

Reports → Stock tab: tap **Low only** to filter down to items running short. The table shows **SKU**, **Name**, **Qty** — a row under threshold is highlighted and marked **— LOW**.



14. Audit log

Audit shows every write anywhere in the system — the register and the back office alike — one row per change: **When**, **Action**, **Entity**, **User**, **Register**, **Payload**.



The figure above shows what a shift's worth of activity looks like end to end: a manager issuing an activation code (`admin.register.code_issue`) and signing in (`admin.login`), then a supervisor's till closing a shift (`shift.close`), taking a payment (`payment.take`), adding order lines (`order.line.add`), and opening the order in the first place (`order.open`) — each one tagged with the till it came from where one was involved.

1. Narrow down with **Entity type** (a dropdown of every recorded kind — Order, Payment, Refund, User, Product, and so on), **Entity id**, **User id**, **Action**, and a **From/To** date range.
2. Tap **Filter** to apply them.
3. Tap **Load more** to bring in the next page (50 rows at a time) without losing the rows already on screen.

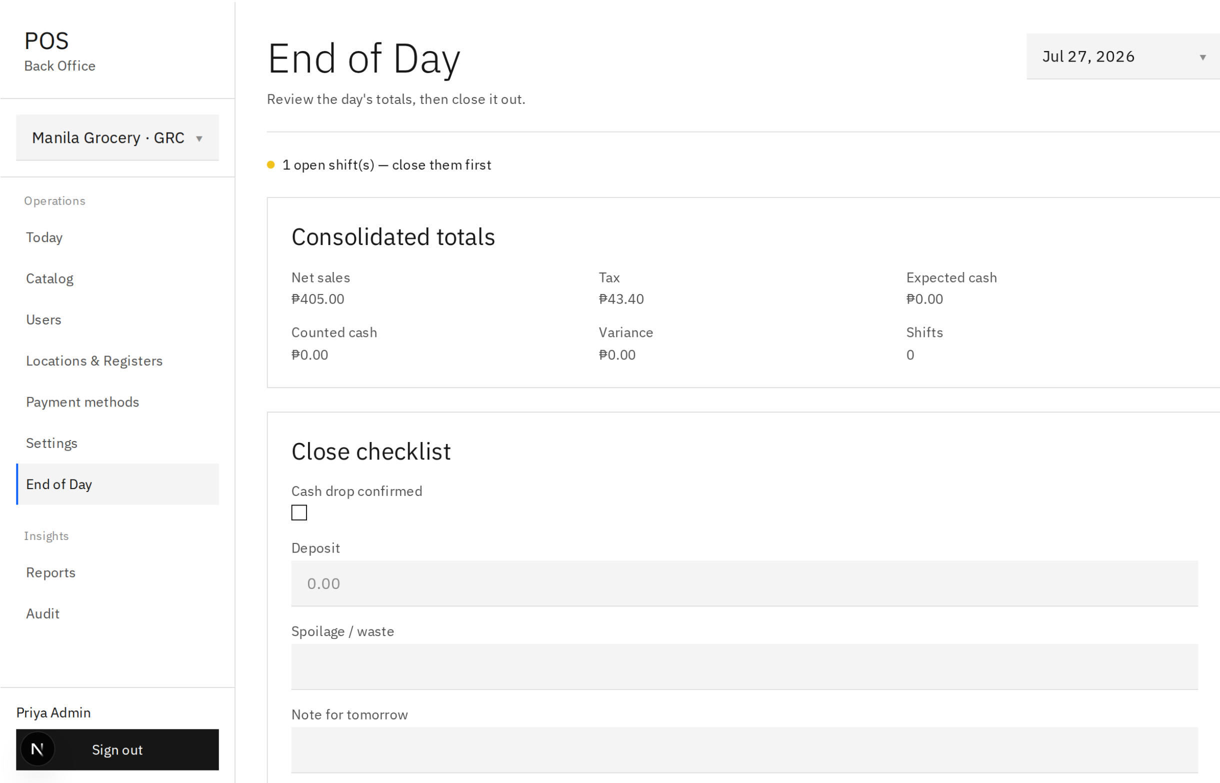
Each row's **Payload** column, where there is one, is collapsed behind a disclosure — tap it to see the raw JSON of what changed. A reprice, for instance, logs the variant's price both **from** and **to**, so "who changed this and what was it before" is always answerable without asking anyone to remember.

This is where a store owner finds out who did what — a repriced variant shows up as `admin.variant.update`, an activation-code issue as `admin.register.code_issue`, a new product as `admin.product.create`, and so on for every entity in the system — not a separate history screen per entity.

15. End of Day

Chapter 7 closed one drawer. This closes the whole store.

End of Day is a back-office section, visible to an admin or anyone granted the **Close business day** permission. It reconciles every till at one location for one date, records the cash going to the bank, and freezes the result. It is the only thing in the system that can stop a shift from opening.



The date is the store's, not yours

The screen always shows one **location** — whichever the sidebar switcher is set to — and one **Business date**, which starts on that location's *own* local today. If you're managing a Manila store from another timezone, the date on this screen is Manila's day, not your browser's. You can look back at an earlier date; you can't pick a later one.

> **Closed up for the night, but tomorrow is still greyed out?** That's deliberate. A business date is the store's *calendar* day, so "tomorrow" only becomes pickable when the store's clock passes midnight — not when you finish closing. Lock the door at 11:17 PM and close the day, and the next date stays greyed out for another 43 minutes. There's nothing you could do there anyway: a day that hasn't started has no shifts, no orders, and all-zero totals. And if it *could*

> be picked tonight, it could be *closed* tonight — and every till would refuse to open in the morning ("The business day is closed…") until an admin reopened it. The greyed-out day is the system protecting tomorrow's opening. It appears on its own at midnight; nothing needs doing before then.

Close the day

1. **End of Day** → confirm the **Business date** is the day you mean.
2. Read the pills along the top. Amber ones are **blockers** — **"N open shift(s) — close them first"** and **"N open order(s)"** — and **Close day** stays greyed out while either stands.
3. Check **Consolidated totals**: **Net sales**, **Tax**, **Expected cash**, **Counted cash**, **Variance**, and the number of **Shifts** the day covered.
4. Fill in the **Close checklist** — tick **Cash drop confirmed**, enter the **Deposit** going to the bank, and add **Spoilage / waste**, a **Note for tomorrow**, or a general **Note**. All of it is optional; an empty checklist still closes the day, and what you left blank is part of the record.
5. Tap **Close day** and confirm.

> **Why cash can read zero while sales don't.** In Figure 15.1 the day has ₱405.00 of net sales but **Expected cash**, **Counted cash**, and **Shifts** all read zero. That's not a fault: the cash columns only count drawers that have actually been *closed and counted*, and the one till here is still open. Sales come from payments as they happen; cash comes from reconciled drawers. Close the till and those figures fill in — which is exactly why an open shift blocks the day close in the first place.

A blue **"N unapproved variance(s)"** pill is a warning, not a blocker. It counts drawers that came up over or short past the approval threshold and haven't been signed off. The day closes anyway, on purpose — the same reasoning as Chapter 7's variance rule: a close that refuses over one unsigned drawer is a close that happens by unplugging something, and then there's no record at all.

What closing does — and what it doesn't

Closing does exactly two things.

- **It freezes the day's numbers.** The totals are copied onto the record as they stood at the moment of close. A refund landing against that date afterwards moves the live reports but never the frozen record — that is the whole point of keeping one.

- **It blocks new shifts.** Anyone opening a drawer at that location on that date is refused: **"The business day is closed. Reopen it before opening a shift."**

Nothing else changes. Refunds, reports, receipts, and variance approvals on that date all still work. Closing a day is a reconciliation milestone, not a lock on the ledger.

Closing is also a once-per-day act. Closing a day that's already closed is refused with **"That day is already closed."** rather than quietly overwriting the record, so a second tap can never rewrite a deposit figure somebody already signed off on.

> **On the totals: Net sales** is ledger-basis — actual payments less actual refunds — while **Tax** is read off the orders closed that day. A refund therefore lowers net sales but not tax. It's the same split behind the day-versus-category question in Chapter 13, and for the same reason: the two numbers answer different questions and aren't meant to reconcile to the cent.

Reopen a closed day

Reopening is **admin only**. A manager holding **Close business day** can close but cannot reopen — reopening is what permits trading again on a date that was already signed off.

1. **End of Day** → pick the closed date. A green **Day closed** pill shows, with the checklist as it was filed.
2. Type a **Reason for reopening**. It's required — **Reopen day** stays disabled until there's something in the box.
3. Tap **Reopen day** and confirm.

Shifts can open on that date again immediately. When the day is closed a second time the totals are re-snapshotted from scratch, so the record reflects whatever happened in between. Every close and every reopen, with its reason, lands in the audit log (Chapter 14).

16. The desktop shell and printing

The register also ships as a desktop app — a Tauri v2 shell that hosts the very same SPA covered in Chapters 4–7, not a third frontend with its own code. What the shell adds on top is a hardware bridge for the two things a browser tab can't do on its own: drive a receipt printer and kick open a cash drawer. The server still decides *what* happens — a sale closes, a receipt is owed — the shell only decides *how* that reaches the hardware in front of it.

Printing

Tap **Print** on a completed sale (Chapter 5) or a closed shift's Z-report (Chapter 7) exactly as you would in a browser tab. What happens next depends on where the till is running:

- In an ordinary browser tab, **Print** opens the browser's own print dialog — nothing about the shell is involved.
- In the desktop shell, **Print** is meant to send the receipt straight to the till's thermal printer instead.

> Note: today, only a **mock** printer driver ships — it writes the receipt out to a file rather than actually driving hardware, so **Print** in the shell doesn't yet put paper through a real printer. A browser tab's print dialog is the one path here that already produces real paper, because it was never the shell's job to begin with.

No-sale drawer opens

The top bar carries a **No sale** button, visible only in the desktop shell (a browser tab has no drawer to pop, so it doesn't show one there) and only if you can open the drawer this way. It's for opening the drawer with nothing being sold — making change for another till, say.

1. Tap **No sale** in the top bar.
2. Type why in **Reason for opening the drawer…** (required).
3. Tap **Open drawer** to confirm — the No-sale form's own confirm button, not the one that starts a shift (Chapter 7); this one only pops the drawer.

The server records who opened it, at which register, and when, before the drawer actually pops — that entry shows up in the back office's audit log (Chapter 14) exactly like a void or a discount does.

17. Troubleshooting

Most of what looks like a bug traces back to one of two boundaries covered earlier in this manual: the split between a device token and a staff session (Chapter 2), or the fact that closing a shift revokes sessions the instant it closes (Chapter 7). The tables below are grouped by where the trouble shows up, not by cause — start with the symptom you're actually looking at.

Activation and sign-in

Symptom	Cause	Fix
Terminal disabled lockout screen appears on a till that was working fine an hour ago	A manager issued this register a fresh activation code. Reissuing revokes the old device token and every staff session bound to it in the same transaction — there's no grace period	Get the new code from **Locations & Registers → Registers → Edit → Issue activation code** (Chapter 11), then type it into the till's own **Activate this terminal** screen (Figure 11.5)
Activation code rejected on **Activate this terminal**	The code has already been redeemed, or it's more than 7 days old — both are refused the same way on purpose	Codes are single-use by design; issue a fresh one from the back office (Chapter 11) rather than retyping the old one
Till stuck on "Enter PIN", nothing happens	Five wrong PINs in a row triggers a 60-second lockout on that till	Wait 60 seconds, then try again (Chapter 3)

Mid-shift, everything suddenly 401s

Symptom	Cause	Fix
Every request from this till starts failing right after a manager issues or reissues this register's activation code	Reissuing kills the old device token and every staff session on it at once — this till is still holding the token that just died	Type the new code into this till's **Activate this terminal** screen, then clock back in with a PIN (Chapter 11)
A cashier's own screen starts failing mid-shift with nothing obviously wrong	Somebody closed this till's shift — closing revokes every staff session bound to that register the instant it closes, not just the one that tapped **Close**	Clock back in with a PIN. If the drawer already shows closed, Chapter 7's Close and count section explains why everyone gets signed out
Approve variance fails immediately and signs the supervisor straight back out to **Enter PIN**	Closing this shift already revoked this register's sessions — the close-shift screen offering that button is running on a session that's already dead, so tapping it can only fail	Approve from a **different, still-open** register at the same location, never the till that just closed — the rule is scoped to the location, not the specific terminal (Chapter 7's Variance and approval)

The Z-report only shows up at close

Symptom	Cause	Fix
Looking for a way to check the Z-report before closing the shift, so the numbers can be reviewed first	There isn't one. The Z-report is part of the close-shift result, not a separate report you pull up beforehand — closing is what produces the figures, and it also revokes the very session that would be showing them	Close the shift; the Z-report and the close result arrive together on the same screen (Chapter 7's "The Z-report — fetched before you close")

The business day is closed

Symptom	Cause	Fix
Open drawer is refused with **"The business day is closed. Reopen it before opening a shift."** — and it's happening on every till at this store at once	A manager closed this location's business day for that date. It's scoped to one location and one date, blocks *only* opening a shift, and no override at the till can lift it	An **admin** reopens it from **End of Day** → pick the date → type a **Reason for reopening** → **Reopen day** (Chapter 15). Shifts open again immediately
Close day is greyed out and won't respond	A blocker is still standing — the amber pills name it: **"N open shift(s) — close them first"** or **"N open order(s)"**. Both must be clear before the day can close	Close the named drawers, and close or transfer any open tab, then come back. An **unapproved variance** pill is blue, not amber — that one is a warning and never blocks the close (Chapter 15)
No **Reopen day** button on a closed day, even though the day clearly needs reopening	Reopening is **admin only**. Holding **Close business day** lets you close a day but deliberately not reopen one — the button doesn't render for a non-admin at all	Ask an admin. If that's you, sign in with the admin account rather than the manager one (Chapter 15)
Closing a day is refused with **"That day is already closed."**	The day was already closed — most often a double-tap, or someone else closed it first. The system refuses rather than overwriting the filed record, so the deposit figure already on file can't be silently rewritten	Nothing to fix. If the filed record genuinely needs changing, an admin reopens the day and closes it again; both actions are logged (Chapter 15)

Catalog at the till

Symptom	Cause	Fix
The till is enrolled and past **Activate this terminal**, but the scan field or menu grid comes up empty	Reading the catalog only needs a valid device token — it doesn't need anyone clocked in. An empty catalog is a real data problem at this location, not a login problem wearing a disguise	Check **Catalog** → **Products** / **Variants** in the back office for active variants at this location (Chapter 9), rather than troubleshooting sign-in

Receipts and printing

Symptom	Cause	Fix
Print in the desktop shell doesn't put any paper through the printer	Only a **mock** printer driver ships in this version — it writes the receipt out to a file instead of driving real hardware	Expected in this version, not a fault to chase; use an ordinary browser tab if paper is actually needed right now (Chapter 16)
Print opens the browser's own print dialog instead of going straight to a receipt printer	Running in an ordinary browser tab rather than the desktop shell — a browser tab has no hardware bridge to a printer at all	By design — the browser's print dialog is the one path here that already produces real paper (Chapter 16)

18. FAQ

Why don't the day and category reports reconcile?

They're not supposed to. **Day** and **User** are **ledger-basis** — summed from actual payments and refunds that moved money. **Category** is **line-basis** — summed from individual order lines, joined to current category names. A payment covers a whole order at once; a category breakdown has to attribute individual lines within it. They answer different questions — "how much cash and card came in" versus "what sold" — so the Day total and the sum of the Category totals aren't expected to match to the cent. See Chapter 13.

Do I have to close the business day every night?

No. Nothing breaks if you don't — orders, shifts, and reports all carry on regardless. Closing is a reconciliation milestone you choose to record: it freezes that date's totals onto a permanent record, captures the bank deposit and the checklist, and stops anyone opening a fresh drawer on a date you've already signed off. Skip it and you simply have no frozen record for that day. See Chapter 15.

Why does a refund change net sales on a closed day but not the tax?

Because the two numbers are read differently, on purpose. **Net sales** is ledger-basis — actual payments taken, less actual refunds paid out. **Tax** is read off the orders that closed that day, and a refund doesn't rewrite the original order (nothing in this system mutates a closed order — Chapter 5). So a refund lowers net sales while the tax figure stays where it was. It's the same split as the day-versus-category question above, and the same answer: the two are answering different questions and aren't meant to reconcile to the cent. See Chapter 15.

What does VAT-inclusive pricing mean for my totals and receipts?

At a location with **Prices include tax** set to **Yes** (all three demo stores are), the shelf price *is* the total — a ₱220.00 item rings up as ₱220.00, full stop. The tax shown on the receipt is **extracted** from that price for the record, not added on top of it. Nothing about the customer's total changes based on tax; only how much of it is labeled tax versus subtotal. See Chapters 2 and 11.

Why can't I delete a product?

Nothing in the back office deletes. Unchecking **Active** and saving **archives** a product, variant, discount, or tax rate instead — it leaves the register's menu, but every past order line, receipt, and report that points at it keeps working, forever. A closed order's lines are a snapshot of name, price, and tax rate at the time of sale, so archiving something never rewrites history. See Chapter 9.

Why did my discount need a supervisor?

Every discount needs one today, regardless of what its own **Requires supervisor** checkbox says in the back office. That flag exists on the Discounts tab, but it's currently inert — the **Discount** button on the till only ever renders for a supervisor's PIN session in the first place, so nothing short of that role can open the panel at all. See Chapter 5.

What happens to an open tab when a register is reissued?

Nothing, to the tab itself. Issuing a new activation code only touches that register's device token and its staff sessions — it never touches orders or shifts. The tab stays open exactly where it was; you simply can't act on it from *that* till until it's re-activated with the new code (Chapter 11). In the meantime it's still visible, and still transferable to another till, from the Floor at that location (Chapter 6).

Can two people use one till at the same time?

Not in the same PIN session. A session belongs to one person, shown by name in the top bar (Chapter 4). **Clock out** ends only that person's session — not the shift, so the drawer and any open tabs stay exactly as they were for whoever clocks in next. Two cashiers share a till across a day by clocking out and back in between them, one at a time, not by both being signed in at once.

19. Glossary

Term	Definition
Till / register	A physical terminal at a location — "till" and "register" mean the same thing throughout this manual (Chapter 1). Its **Mode** (retail or food) decides which sale screen it shows (Chapters 4, 11).
Device token	The long-lived credential a terminal holds once activated, proving *which register* a request came from. Alone, it can read the catalog; it can never touch money (Chapters 2, 3).
Activation code	The one-time, 7-day, admin-issued credential a till exchanges for its device token. Single-use — never typed in twice (Chapters 3, 11).
Staff session	The short-lived credential a PIN clock-in produces, proving *who* is acting on an already-authenticated register. Bound to that specific register — lifted to another till, it's inert (Chapters 2, 3).
PIN	The 4–6 digit code a cashier or supervisor types to start a staff session. Five wrong attempts in a row locks that till for 60 seconds (Chapter 3).
Shift	A till's own cash-accountability period: opens with a counted float, runs sales and cash movements, closes with another count (Chapter 7).
Float	The cash counted into the drawer when a shift opens, before a single sale (Chapter 7).
Cash movement	Any cash that moves in or out of the drawer without a sale attached — a payout, a deposit, a paid-in — always recorded with a reason (Chapter 7).
Z-report	The sales-and-cash summary produced the moment a shift closes: totals by tender, **Paid in**, **Payouts**, **Drops**, and order counts. It only ever appears together with the close itself (Chapter 7).
Variance	The difference between counted cash and expected cash at close. Inside the store's threshold it reconciles clean; outside it, a supervisor has to approve it from a different register at the same location (Chapter 7).
Tab	An open order at a food-service till — seated at a table or held for a walk-in — that lingers before payment instead of closing in under a minute (Chapter 6).
`table_ref`	The optional table name or number attached to a tab; left blank for a walk-in or a bar tab (Chapter 6).
Course / fire	The prep-state chip on a kitchen-tracked line — **Pending** → **Cooking** → **Ready**. Tapping it moves the line forward; **Cooking** is the "fired" state the kitchen sees (Chapter 6).
Modifier	A single pick attached to a menu item — an extra, a substitution, an option — priced as a delta on the item's own price (Chapter 6).
Modifier group	A named set of modifiers attached to a product (e.g. **Rice**, **Add-ons**). Can be marked required, and a repeat-legal choice can be added more than once (Chapters 6, 9).
Signed delta	

Term	Definition
	A modifier's price relative to the item it's attached to — usually positive (an add-on), sometimes negative, like **No Rice** subtracting from Chicken Adobo (Chapter 6).
Split	Dividing one tab's payment across 2 to 10 separate checks once payment has started. The checks always sum back exactly to the original total (Chapter 6).
Transfer	Moving an open tab from one register to another. The receiving till's shift owns the sale's money from the instant it lands (Chapter 6).
Restock	Putting a refunded line's quantity back into on-hand stock. Left checked by default; cleared if the returned goods are damaged (Chapter 5).
Payment method	A named way of paying at one location: Cash, Visa, GCash. It belongs to a payment method group, which is what decides how it behaves. Its code is fixed once created; its name isn't (Chapter 12).
Payment method group	A kind of tender at one location — Cash, Cards, E-wallets — holding one or more payment methods. The group decides the behaviour: a cash group opens the drawer and computes change, every other kind only records what another machine did and can't be refunded through the till (Chapter 12).
Ledger basis	A report built from actual payments and refunds that moved money — the Sales report's **Day** and **User** tabs (Chapter 13).
Line basis	A report built from individual order lines, joined to current catalog data — the Sales report's **Category** tab. Doesn't reconcile with a ledger-basis report, by design (Chapter 13).
VAT-inclusive	A location's **Prices include tax** setting: the shelf price already contains tax, and tax on the receipt is extracted from that price rather than added on top (Chapters 2, 11).
SKU	A variant's own stock-keeping code — distinct from its barcode. Every variant needs one (Chapter 9).
Barcode	The optional code a scanner reads to find a variant directly. A variant needs no barcode to exist, but a scan needs one to find it (Chapter 9).
Archive	The back office's only removal action: unchecking **Active** on a product, variant, discount, or tax rate takes it off the live catalog while every past order line, receipt, and report that points at it keeps working (Chapter 9).